

PYTER ÁVILA

**Cãomunidade - Uma plataforma
tecnológica para gestão de cães
comunitários**

Trabalho de Conclusão de Curso
apresentado como requisito parcial
para a obtenção do grau de Tecnólogo em
Análise e Desenvolvimento de Sistemas

Prof. Rafael Betito
Orientador

Rio Grande, dezembro de 2025

AGRADECIMENTOS

Agradeço imensamente a minha companheira Livia Cunha Serpa, que sempre me incentivou a seguir em frente com o curso desde quando começamos a nos relacionar. Sem ela, nada disso seria possível.

Agradeço profundamente meus cãesinhos: Dara (*in memoriam*), Dexter e Phoebe, e também a minha gata Frida por me inspirarem a escolher a motivação deste trabalho. Meu amor e gratidão todo a eles.

Agradeço a minha sogra Cátia Eugênia Cunha, pelo seu papel crucial na minha jornada, me apoiando e me incentivando a seguir em frente.

Agradeço aos meus amigos, tanto os que fiz dentro do curso como os que sempre estiveram comigo, por me acompanharem nessa jornada. Agradeço também ao meu orientador, Rafael Betito, pela paciência e sabedoria em me guiar na construção deste trabalho. Agradeço também aos professores do Instituto, pelo ensino de qualidade ao longo dos anos.

SUMÁRIO

LISTA DE FIGURAS	5
LISTA DE TABELAS	7
LISTA DE CÓDIGOS	8
LISTA DE SIGLAS	9
1 INTRODUÇÃO	10
1.1 Objetivos	11
1.1.1 Objetivo Geral	11
1.1.2 Objetivos Específicos	11
2 SISTEMAS EXISTENTES	12
2.1 Pet Coletivo	12
2.2 Viu meu Pet	14
2.3 Vakinha	18
2.4 Tabela comparativa	19
3 ANÁLISE	21
3.1 Elicitação de Requisitos	21
3.1.1 Requisitos Funcionais	21
3.1.2 Requisitos Não Funcionais	26
3.2 Diagrama de Casos de Uso	27
3.2.1 Casos de Uso	27
3.3 Diagrama de Atividades	28
3.3.1 Criar Pedido de Ajuda Financeiro	29
3.3.2 Realizar Doação via PIX	29
3.3.3 Realizar Saque via PIX	30
3.4 Prototipação de Tela	31
3.4.1 Tela de Avistamentos	31
4 ARQUITETURA E PROJETO	35
4.1 Diagrama de Arquitetura	35
4.2 Diagrama de Classes	35
4.3 Diagrama de Documentos	36

5	IMPLEMENTAÇÃO	40
5.1	Ferramentas	40
5.1.1	React Native	40
5.1.2	Expo	41
5.1.3	Firebase	41
5.1.4	AbacatePay	42
5.2	Desenvolvimento	43
5.2.1	Function de Criação de PIX do AbacatePay	43
5.2.2	Webhook de Confirmar Pagamento	45
5.2.3	Envio de Notificações	48
5.2.4	Renderização de Mensagens	50
6	TESTES	55
6.1	Cadastro de um Cão	55
6.2	Alerta de Cão Desaparecido	57
6.3	Cadastro de um Avistamento	59
6.4	Pedido de Ajuda Financeiro	60
6.5	Multiplataforma	64
7	CONCLUSÃO	67
	REFERÊNCIAS	69

LISTA DE FIGURAS

1.1	Percentagem de cães e gatos em situações de rua no Brasil. Fonte: (Mars Petcare, 2024)	10
2.1	Listagem de Campanhas do Pet Coletivo. Fonte: (Pet Coletivo, 2025a)	12
2.2	Perfil de uma Campanha do Pet Coletivo. Fonte: (Pet Coletivo, 2025a)	13
2.3	Dados de uma Campanha do Pet Coletivo. Fonte: (Pet Coletivo, 2025a)	13
2.4	Comprovantes de Despesa de uma Campanha do Pet Coletivo. Fonte: (Pet Coletivo, 2025a)	13
2.5	Página Inicial do Viu Meu Pet. Fonte: (Viu meu Pet, 2025)	14
2.6	Página Inicial do Viu Meu Pet - Aba de adoção. Fonte: (Viu meu Pet, 2025)	14
2.7	Listagem de animais para adoção do Viu Meu Pet. Fonte: (Viu meu Pet, 2025)	15
2.8	Listagem de animais perdidos do Viu Meu Pet. Fonte: (Viu meu Pet, 2025)	15
2.9	Perfil de um anúncio de adoção no Viu Meu Pet (1). Fonte: (Viu meu Pet, 2025)	16
2.10	Perfil de um anúncio de adoção no Viu Meu Pet (2). Fonte: (Viu meu Pet, 2025)	16
2.11	Histórias de Sucesso do Viu Meu Pet	17
2.12	Aba de Adotados do Viu Meu Pet	17
2.13	Página inicial das campanhas de arrecadação do Vakinha. Fonte:(Vakinha, 2025)	18
2.14	Perfil de uma campanha de arrecadação do Vakinha. Fonte:(Vakinha, 2025)	18
2.15	Perfil de uma campanha de arrecadação do Vakinha. Fonte:(Vakinha, 2025)	19
3.1	Diagrama de Casos de Uso	28
3.2	Diagrama de Atividade - Criar Pedido de Ajuda Financeiro	29
3.3	Diagrama de Atividade - Realizar Doação via PIX	30
3.4	Diagrama de Atividade - Realizar Saque via PIX	31
3.5	Protótipo - Tela de Avistamentos	32
3.6	Protótipo - Tela de Cães	32
3.7	Protótipo - Tela de Eventos	33
3.8	Protótipo - Tela de Pedidos de ajuda	33
3.9	Protótipo - Tela de Pedidos de ajuda	34

4.1	Diagrama de Arquitetura	35
4.2	Diagrama de Classes	36
4.3	Diagrama de Documentos	36
4.4	Coleção users	37
4.5	Coleção caes	37
4.6	Coleção eventos	38
4.7	Coleção pedidos_ajuda	39
4.8	Coleção chats	39
4.9	Coleção tags	39
6.1	Formulário de cadastro de um Cão preenchido	55
6.2	Perfil do novo Cão cadastrado	56
6.3	Listagem de Cães com o novo Cão cadastrado	56
6.4	Registro do novo cão no Firestore	57
6.5	Evento de Cão Desaparecido e notificação sobre	57
6.6	Perfil de um cão desaparecido	58
6.7	Registro do novo cão no Firestore	58
6.8	Tela de Avistamentos	59
6.9	Registro do Avistamento de um cão	60
6.10	Registro do Avistamento de um cão	60
6.11	Formulário de cadastro de um pedido de ajuda	61
6.12	Pedido de ajuda financeiro - Visão do criador do pedido	62
6.13	Pedido de ajuda financeiro - Visão do doador do pedido	62
6.14	Modal de doação	63
6.15	Cobrança pendente no AbacatePay	63
6.16	Doação pendente no Firestore	63
6.17	Cobrança paga no AbacatePay	64
6.18	Doação paga no Firestore	64
6.19	Prestação de contas do pedido de ajuda financeiro	64
6.20	Usuário na plataforma Android acessando um pedido de ajuda	65
6.21	Usuário na plataforma iOS trocando mensagens no chat com o usuário da plataforma Android	66
6.22	Usuário na plataforma Android trocando mensagens no chat com o usuário da plataforma iOS	66

LISTA DE TABELAS

2.1	Comparativo entre os sistemas existentes	20
7.1	Comparativo entre os sistemas existentes e o Cãomunidade	67

LISTA DE CÓDIGOS

5.1	Function de criação de PIX do AbacatePay	43
5.2	Corpo da requisição para criar o PIX	44
5.3	Resposta de sucesso da criação do PIX	44
5.4	Corpo de requisição enviado ao Webhook	45
5.5	Autenticação da chave enviada pelo Webhook	45
5.6	Atualização de status de pagamento de um PIX	46
5.7	Gatilho do Firestore para quando um novo Evento de Cão é cadastrado . .	48
5.8	Function usada para obter tokens de seguidores de um cão	49
5.9	Function de enviar notificações <i>push</i>	49
5.10	Íncio do código do Chat	50
5.11	Declaração de estados e primeiro useEffect do Chat	51
5.12	Segundo useEffect do Chat e função chamada ao digitar mensagem	51
5.13	Funções de envio e de gerenciamento de estado de texto digitado pelo usuário	52
5.14	Código dos componentes que são renderizados em tela	53

LISTA DE SIGLAS

API	<i>Application Programming Interface</i>
GPS	<i>Global Positioning System</i>
IA	Inteligência Artificial
JSON	<i>JavaScript Object Notation</i>
MVP	<i>Minimum Viable Product</i>
NoSQL	<i>Not Only SQL</i>
PDF	<i>Portable Document Format</i>
PIX	Sistema de Pagamentos Instantâneos
RF	Requisitos Funcionais
RNF	Requisitos Não Funcionais
SRD	Sem Raça Definida
TCC	Trabalho de Conclusão de Curso
URL	<i>Uniform Resource Locator</i>

1 INTRODUÇÃO

No Brasil, segundo dados coletados de 2023 a 2024 em um estudo realizado pela Mars Petcare (Mars Petcare, 2024), existem cerca de oitenta e dois milhões de cães no país. Destes oitenta e dois milhões, vinte milhões de cães do país estão em situação de rua, e apenas cento e setenta e sete mil estão em abrigos comunitários, conforme descrito em gráficos na Figura 1.1.

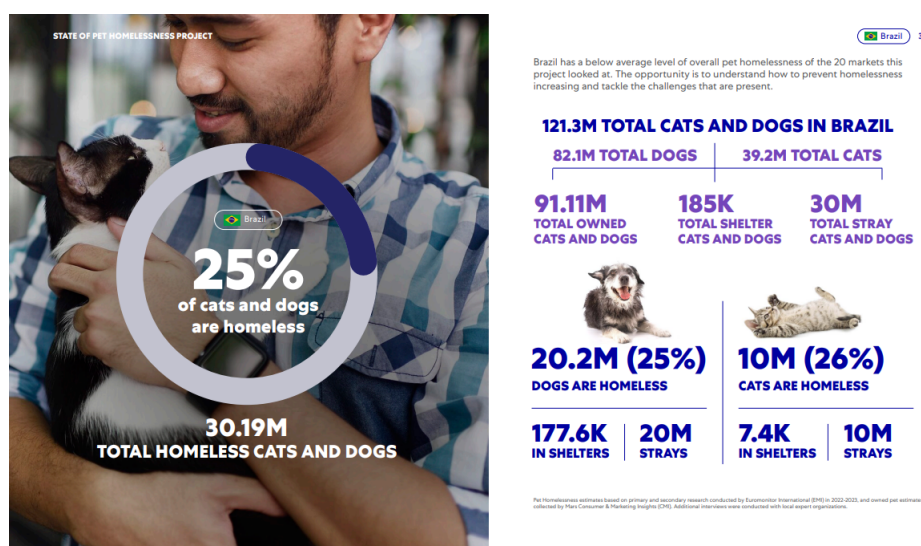


Figura 1.1: Percentagem de cães e gatos em situações de rua no Brasil. Fonte: (Mars Petcare, 2024)

Existem pessoas que querem ajudar a tornar estes números mais favoráveis, principalmente na cidade de Rio Grande, seja apoiando voluntários com recursos financeiros, cuidados médicos ou fornecendo um lar, mesmo que temporário.

A mobilização utilizada atualmente para o cuidado de cães na cidade de Rio Grande é descentralizada, sendo necessário para que qualquer pessoa interessada em contribuir com cuidados para os cães comunitários necessite acessar diversos grupos e voluntários. Segundo voluntários das comunidades de cuidadores de cães, um *software* que centralizasse as informações necessárias para auxiliar nos cuidados dos cães seria de grande ajuda.

Informações de um cão, como quais procedimentos médicos ele já fez, se ele é microchipado ou até mesmo o local onde o mesmo se encontrava em determinado dia e hora, são úteis em todos os casos em que um cão de rua desaparece. A centralização de campanhas de arrecadação e pedidos de ajuda é importante pois somente quem está

por dentro das comunidades tem acesso as suas necessidades, criando uma barreira social para quem quer contribuir com os cuidados sem precisar estar a par de cada grupo de uma comunidade de voluntários específica.

Um outro obstáculo para atrair doadores para as campanhas é a transparência na utilização do dinheiro arrecadado. Muitas pessoas acreditam que campanhas de arrecadação para cuidados de cães comunitários podem passar como golpes, sem uma clara indicação da onde aquele dinheiro é utilizado. A prestação de contas destas campanhas se torna uma necessidade para incentivar o auxílio financeiro com os cuidados de cães comunitários.

Embora já existam plataformas que proveem soluções tecnológicas para os problemas citados, nenhuma delas centraliza todos, fazendo com que usuários tenham que acessar cada plataforma para algum problema específico.

Levando isso em conta, esse projeto se propõe a criar um aplicativo que centralize todas as soluções para esse problemas, providenciando funcionalidades suficientes para entregar uma plataforma ágil, transparente, informativa e de fácil acesso, para quaisquer pessoas que queiram, ou já ajudam nos cuidados de cães comunitários.

1.1 Objetivos

Este projeto tem como objetivo criar uma plataforma tecnológica que visa auxiliar no cuidado de cães comunitários, providenciando um canal entre doadores e receptores de campanhas de arrecadação, uma maneira de catalogar os cães, atualizar as situações dos mesmos e rastrear cães desaparecidos.

1.1.1 Objetivo Geral

Desenvolver um aplicativo que entregue soluções tecnológicas centralizadas para a resolução de problemas relacionados aos cuidados de cães comunitários.

1.1.2 Objetivos Específicos

Os objetivos específicos determinados para alcançar o sucesso na implementação do sistema são os seguintes:

- Ser Multiplataforma;
- Ter um cadastro individual para cada cão;
- Possuir um mapa de Avistamentos, que registram com uma foto e uma data aonde um determinado cão foi visto;
- Pedidos de ajuda não financeiros, para quando é necessário cuidados com um cão onde recursos financeiros não são necessários;
- Campanhas de arrecadação;
- Eventos de cães, que são enviados como notificações para usuários que seguem o cão;
- *Chat* integrado ao aplicativo, permitindo a comunicação dentro do sistema.

2 SISTEMAS EXISTENTES

Neste capítulo, serão apresentados os sistemas que foram utilizados para comparação com o Cãomunidade.

Após uma extensa pesquisa, foram selecionados três sistemas diferentes, que compartilham dos mesmos objetivos ou possuem funcionalidades semelhantes ao Cãomunidade. São eles: **Pet Coletivo**, **Viu meu Pet** e **Vakinha**.

2.1 Pet Coletivo

Pet Coletivo (Pet Coletivo, 2025a) é um sistema disponível em ambos Android e Web em que o foco é a criação de campanhas de arrecadação e rifas para cuidados de *pets*, sejam eles comunitários ou até mesmo adotados.

As campanhas podem ser de arrecadação, com uma meta para se alcançar, ou rifas, que são semelhantes as campanhas de arrecadação, mas permitem que quem doe escolha números da rifa e concorra a prêmios da rifa.

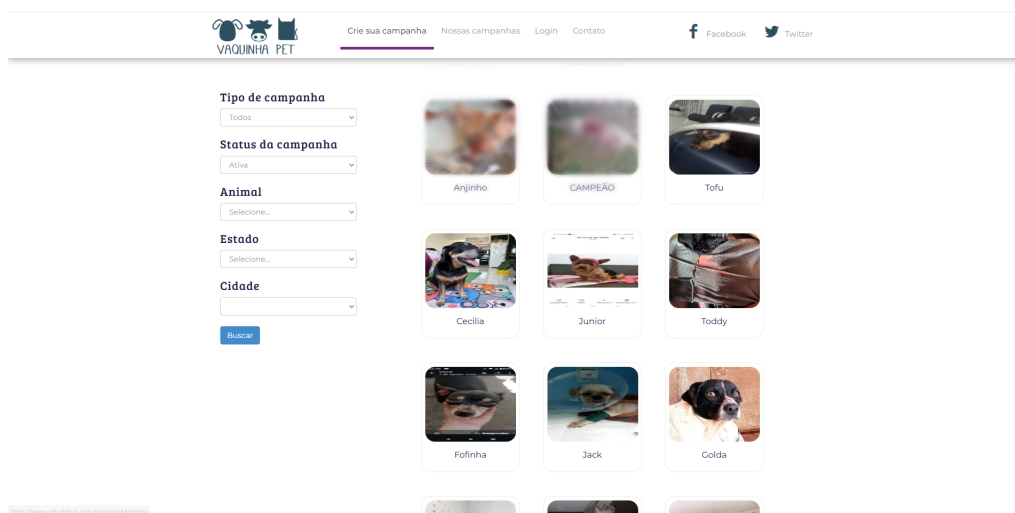


Figura 2.1: Listagem de Campanhas do Pet Coletivo. Fonte: (Pet Coletivo, 2025a)

Nas campanhas é possível informar a meta de doação, o nome do animal que está sendo auxiliado com a campanha e uma breve descrição sobre a campanha em si, contando mais da história do animal e qual a sua necessidade.

No perfil de uma campanha também é possível ver, conforme a Figura 2.3, o valor que foi arrecadado, o valor que está pendente, quando a campanha foi criada, qual a raça do animal, quem é o responsável pela campanha, qual é a meta, fotos de antes da campanha,

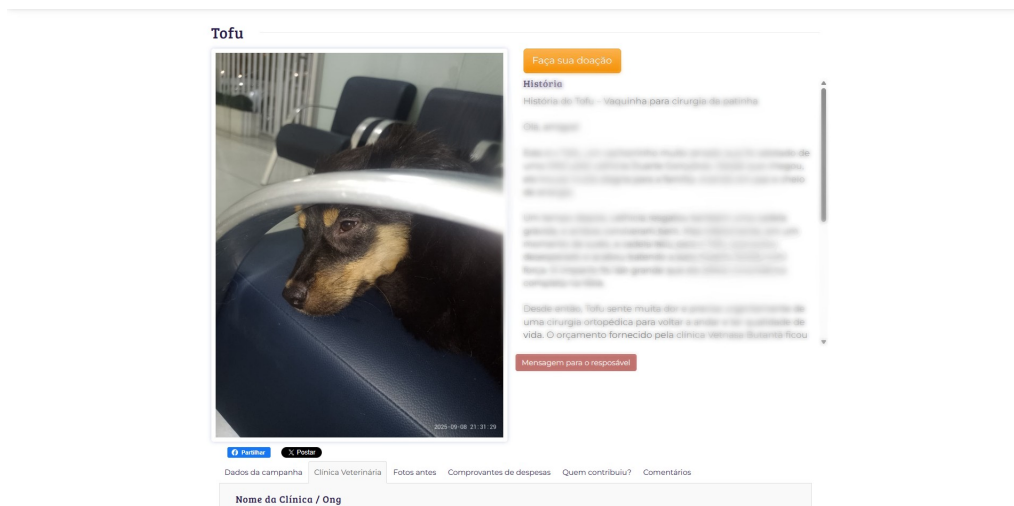


Figura 2.2: Perfil de uma Campanha do Pet Coletivo. Fonte: (Pet Coletivo, 2025a)

quem contribuiu, qual a clínica veterinária que irá realizar os procedimentos da campanha (se houver procedimentos cirúrgicos necessários) e comprovantes de despesa, como pode ser visto na Figura 2.4, que podem ser desde notas fiscais ou outros comprovantes de pagamento, até fotos comprovando que algum procedimento médico necessário foi feito, ou até compras de mantimentos.

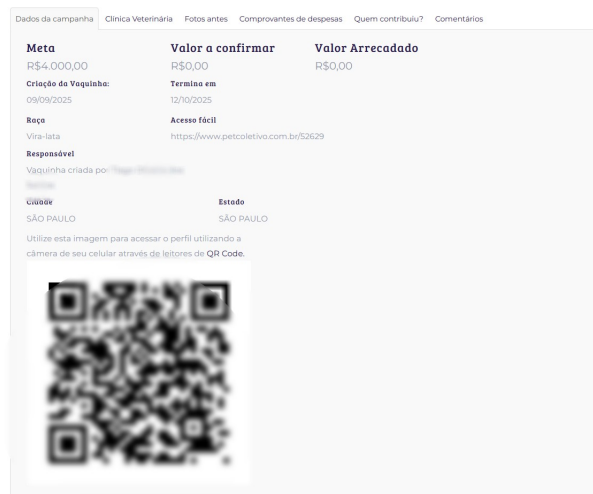


Figura 2.3: Dados de uma Campanha do Pet Coletivo. Fonte: (Pet Coletivo, 2025a)



Figura 2.4: Comprovantes de Despesa de uma Campanha do Pet Coletivo. Fonte: (Pet Coletivo, 2025a)

O Pet Coletivo permite que as doações para campanhas sejam feitas via cartão de crédito ou boleto, usando o *gateway* de pagamento do PagSeguro e cobrando uma taxa de

cinco vírgula noventa e nove por cento em cima do valor da contribuição (Pet Coletivo, 2025b).

2.2 Viu meu Pet

Viu meu Pet (Viu meu Pet, 2025) é um sistema disponível somente para *Web* que tem como foco principal auxiliar tutores que perderam seus animais ou que querem adotar/anunciar para adoção algum animal.



Figura 2.5: Página Inicial do Viu Meu Pet. Fonte: (Viu meu Pet, 2025)

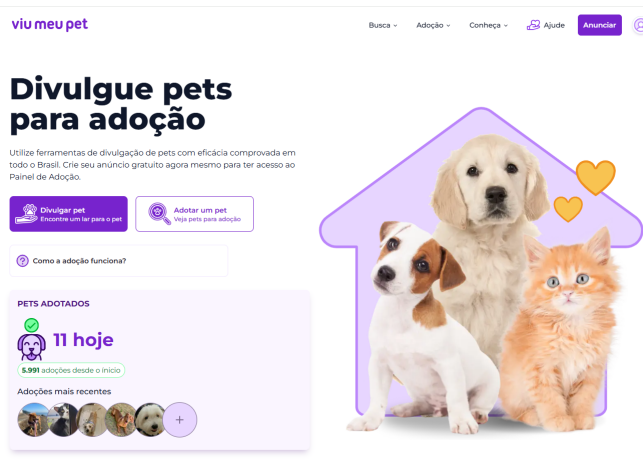


Figura 2.6: Página Inicial do Viu Meu Pet - Aba de adoção. Fonte: (Viu meu Pet, 2025)

O sistema contém uma aba de adoção, uma aba de achados e outra aba de perdidos. O sistema conta também com uma opção de doação, mas não para campanhas específicas, as doações feitas para o *site* são de forma geral.

Existe uma funcionalidade de prestação de contas, mas que até o momento não foi implementada.

Na listagem de animais para adoção e animais perdidos, apresentadas respectivamente nas Figuras 2.7 e 2.8, é possível filtrar por espécie do animal, gênero, porte, cor predominante, cor dos olhos, raça, idade e distância máxima, partindo da localização do usuário ou do endereço que o usuário esteja filtrando também.

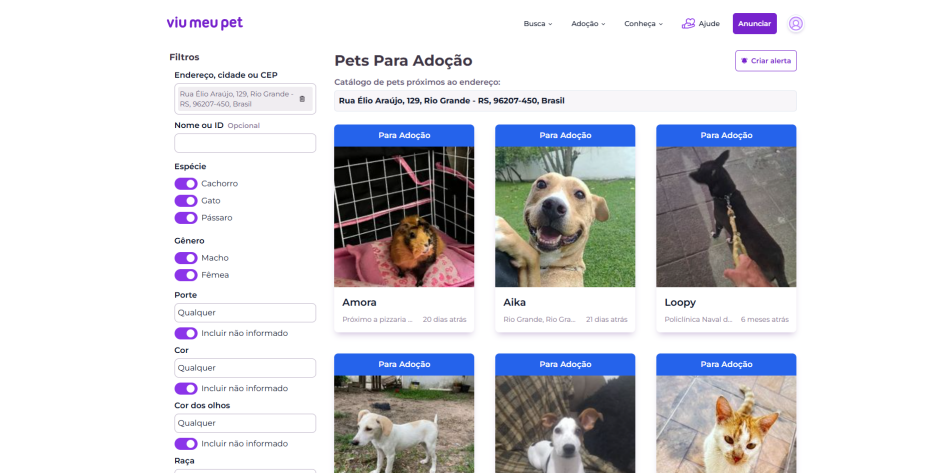


Figura 2.7: Listagem de animais para adoção do Viu Meu Pet. Fonte: (Viu meu Pet, 2025)

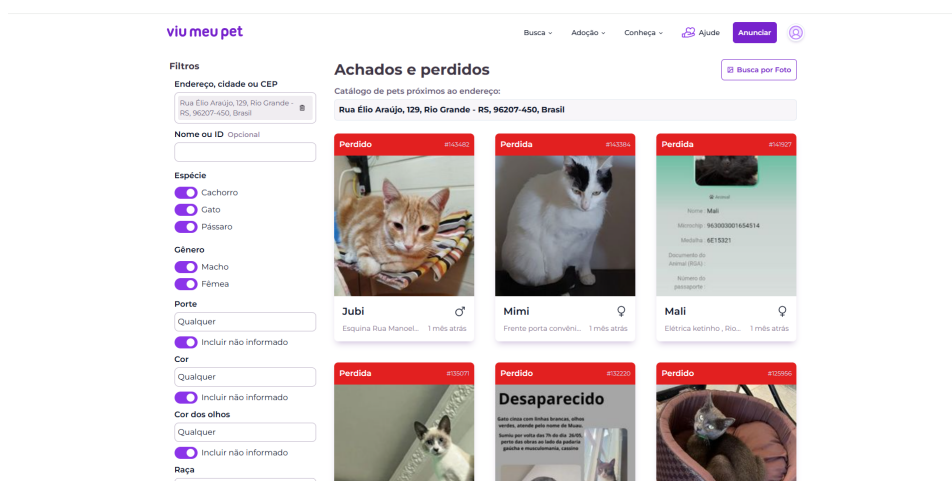


Figura 2.8: Listagem de animais perdidos do Viu Meu Pet. Fonte: (Viu meu Pet, 2025)

O perfil de um animal, apresentado em ambas as Figuras 2.9 e 2.10, é composto por uma breve descrição do animal, informações de seu anunciante, seja um anúncio de adoção ou de desaparecimento, suas características de raça, idade, gênero, porte, cor predominante e cor dos olhos, e também da data e local que ele foi anunciado ou que desapareceu.

Quando um animal é encontrado ou adotado, seu anúncio é atualizado, com a data em que foi encontrado e o local ou com a data em que foi adotado.

O Viu Meu Pet conta também com a aba de Histórias de Sucesso, apresentada na Figura 2.11, que é semelhante a aba de Adotados, presente na Figura 2.12, demonstrando quais animais já foram encontrados ou adotados.

Ambas abas contam com o mesmo perfil disponível para os animais disponíveis para adoção ou perdidos, mas com as informações atualizadas conforme o seu *status*, com os animais que foram adotados constando na data quando eles foram adotados e os animais reencontrados constando a data de quando foram encontrados.

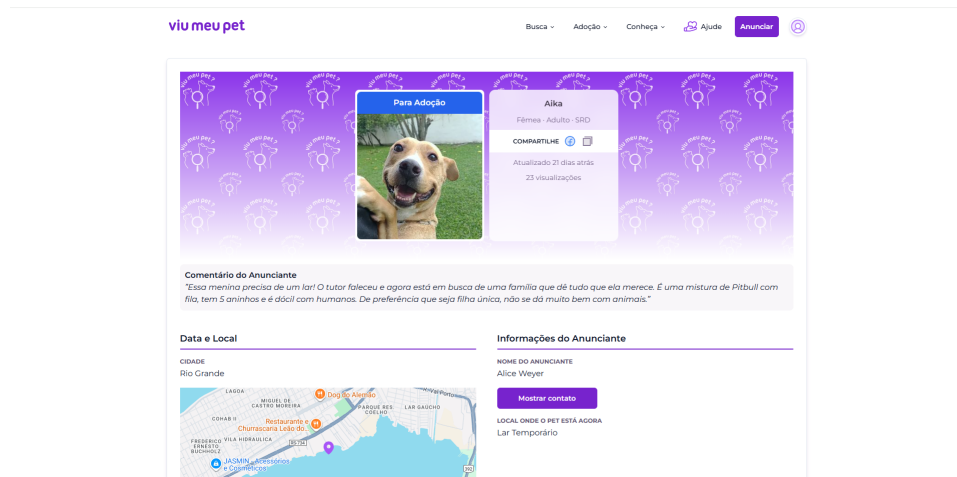


Figura 2.9: Perfil de um anúncio de adoção no Viu Meu Pet (1). Fonte: (Viu meu Pet, 2025)

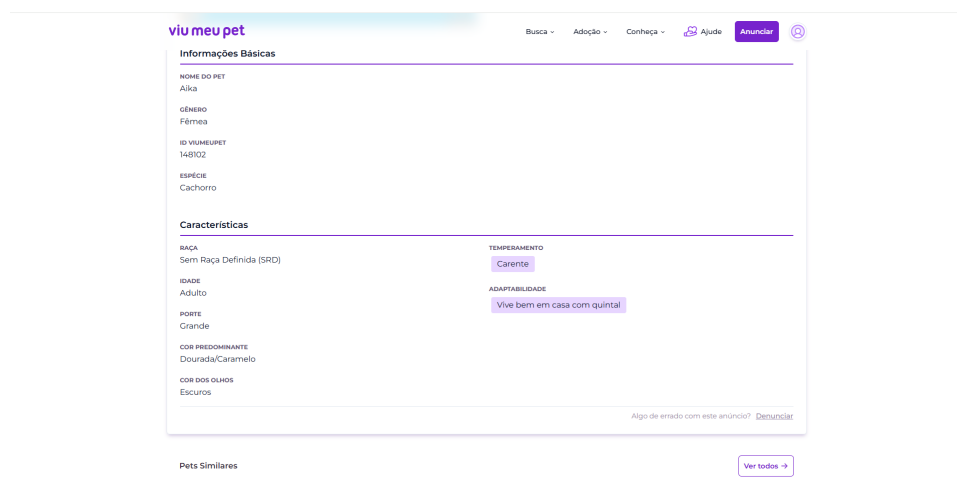


Figura 2.10: Perfil de um anúncio de adoção no Viu Meu Pet (2). Fonte: (Viu meu Pet, 2025)

viu meu pet

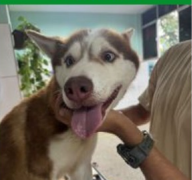
Busca ▾ Adoção ▾ Conheça ▾ Ajude Anunciar 

Últimos Reencontros

Veja as histórias de sucesso de pets que foram resgatados e voltaram para casa.

7.703 reencontros até o momento.

Tutor Encontrado



Tutor Encontrado após 8h 24m

Nome desconhecido

Paralela, Salvador 5 horas atrás

Encontrada



Encontrada após 14h 37m

Milly

Vila Carvalho, Soroc... 6 horas atrás

Encontrada

Procurar-se
Atende pelo nome de Bela,
Entrar com contato pelo
watzap 6



Encontrada após 2h 52m

Bella

Próximo a APAE, Lot... 8 horas atrás

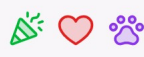
Encontrado



Encontrado após 1d 11h

Tufão

Ceilandia, Ceilandia 13 horas atrás



História em destaque



Reencontro de
Tico

Tais ★★★★★

Belo Horizonte - BH

Figura 2.11: Histórias de Sucesso do Viu Meu Pet

viu meu pet

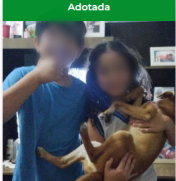
Busca ▾ Adoção ▾ Conheça ▾ Ajude Anunciar 

Últimas Adoções

Veja as histórias de sucesso de pets que foram adotados e ganharam um novo lar.

5.991 pets adotados até o momento.

Adotada

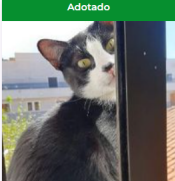


Adotada após 15d 1h

Luna

Perto do Mercado d... 3 horas atrás

Adotado

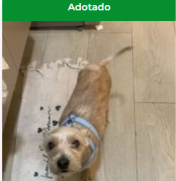


Adotado após 5d 22h

Frajola

Leonel de Azevedo 5 horas atrás

Adotado

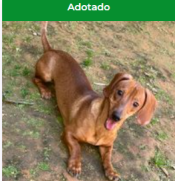


Adotado após 4d 14h

Fiapo Provisório

Próximo a praça da ... 6 horas atrás

Adotado



Adotado após 14h 5m

Luke

Próximo a escola Lui... 7 horas atrás



História em destaque

Conheça mais uma das **centenas de histórias** que aquecem corações e fazem todo o nosso trabalho valer a pena!



Adoção de
Atena

Victor ★★★★★

São Paulo - SP

"Felizmente Atena conseguiu uma família, e isto rapidamente, o site o qual eu não conhecia, mas agora recomendo e muito, teve um alcance. Ao resgata-la disse que acharia um dono em dois dias, pois minha família estava muito descontente ainda que eu tenha afirmado

Figura 2.12: Aba de Adotados do Viu Meu Pet

2.3 Vakinha

O sistema Vakinha (Vakinha, 2025) é uma plataforma disponível para *Web*, *Android* e *iOS* que é dedicada á criação de campanhas de arrecadação, que são chamadas de "vakinhinhas".

As vakinhinhas são separadas em diversas categorias, uma delas sendo *Animais/Pets*.



Figura 2.13: Página inicial das campanhas de arrecadação do Vakinha. Fonte:(Vakinha, 2025)

As campanhas de arrecadação, conforme apresentado na Figura 2.14, possuem diversas informações, como o valor que foi arrecadado até o momento, quantas pessoas apoiaram e uma breve descrição sobre a campanha.



Figura 2.14: Perfil de uma campanha de arrecadação do Vakinha. Fonte:(Vakinha, 2025)

As campanhas de arrecadação possuem também uma aba de atualizações, que permite que o usuário criador da campanha cadastre atualizações sobre a mesma, como quanto dinheiro já foi sacado, o que foi feito com o dinheiro, troca de meta, dentre outros tipos de atualizações, conforme é demonstrado na Figura 2.15

É possível também virar um Promotor do Bem, ao compartilhar o *link* de uma campanha de arrecadação. Um Promotor do Bem pode ver quantas pessoas doaram para a campanha que ele compartilhou o *link*.

Semelhante ao Promotor do Bem é a funcionalidade de adotar uma campanha, que é uma funcionalidade que permite que um usuário divulgue uma campanha no dia de seu aniversário, pedindo doações para a campanha como uma forma de presentear-lo.

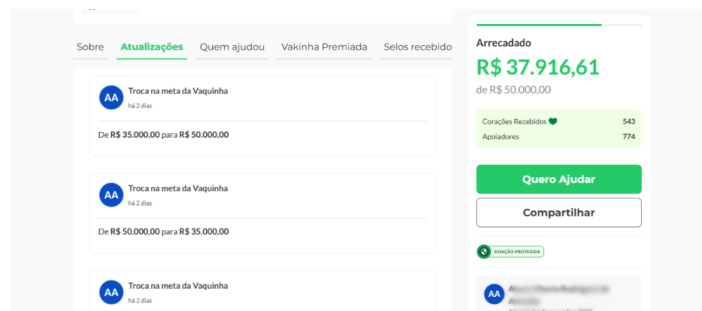


Figura 2.15: Perfil de uma campanha de arrecadação do Vakinha. Fonte:(Vakinha, 2025)

2.4 Tabela comparativa

A Tabela 2.1 apresenta um resumo comparativo entre as funcionalidades dos sistemas analisados e a solução proposta pelo Cãomunidade.

Diversas funcionalidades foram especificadas e analisadas, como a funcionalidade **Plataforma**, onde foi analisado em quais plataformas o sistema esta disponível.

O **Cadastro de Cães** foi outro aspecto analisado, visando entender como cada sistema trata o cadastro de um cão e se ele sequer existe. Foi analisado se o sistema individualiza o cadastro do cão de seus objetivos, como ser encontrado ou adotado, afim de disponibilizar um perfil para eventos posteriores do cão.

O **Mapa de Avistamentos** é a funcionalidade responsável por providenciar um mapa contendo as últimas localidades onde um cão foi avistado, afim de rastrear sua movimentação, em casos que seja necessário consultar o histórico por onde um cão que desapareceu esteve.

A funcionalidade de **Campanhas Financeiras** foi especificada para analisar se o sistema usado de comparação possui alguma forma de criação de uma campanha para arrecadar fundos, com algum objetivo, como pagar uma cirurgia ou consulta de um veterinário.

A **Prestação de Contas** é uma funcionalidade que, na maioria dos casos, é usada para complementar a de campanhas. Foi analisado se os sistemas existentes providenciam alguma forma de identificar para onde o dinheiro arrecadado em uma campanha é utilizado.

Foi analisado também se os sistemas possuem alguma forma de **Alertas de Desaparecimento**, notificando de alguma maneira os seus usuários de quando um cão desaparece.

E por fim, foi analisado se os sistemas providenciavam alguma maneira de dois usuários se comunicarem por dentro do sistema em si, algo como um **Chat Integrado**, sem necessidade e aplicativos terceiros para combinar alguma coisa relacionada aos cuidados de um cão.

Tabela 2.1: Comparativo entre os sistemas existentes

Funcionalidade	Pet Coletivo	Viu meu Pet	Vakinha
Plataforma	<i>Web / Android</i>	<i>Web</i>	<i>Web</i>
Cadastro individual de Cães	Não	Não	Não
Mapa de Avistamentos	Não	Não	Não
Campanhas Financeiras	Sim	Não	Sim
Prestação de Contas	Sim	Não*	Sim
Alertas de Desaparecimento	Não	Sim	Não
Chat Integrado	Não	Não	Não

*Funcionalidade prevista mas não implementada no sistema citado).

3 ANÁLISE

3.1 Elicitação de Requisitos

Neste capítulo, apresentamos o processo de eliciação de requisitos que guiou a definição das funcionalidades do sistema. A eliciação de requisitos é uma etapa fundamental no desenvolvimento de *software*. De acordo com Pressman (PRESSMAN; MAXIM, 2021), essa etapa visa coletar e analisar as necessidades dos usuários finais para garantir que o sistema atenda às expectativas e requisitos do seu público-alvo.

Para a definição das funcionalidades do sistema, foi realizada uma entrevista semiestruturada com uma voluntária apenas da Bicharada Universitária (grupo de voluntários da FURG). A entrevista foi feita dia 04/09/2025 com uma voluntária e durou por volta de 36 minutos. Essa voluntária compartilhou sua experiência no dia a dia do cuidado de cães e suas dificuldades encontradas na gestão dos mesmos. A partir dessa conversa, foram levantados os principais requisitos funcionais e não funcionais, que em seguida foram documentados e priorizados para orientar o desenvolvimento do sistema.

As funcionalidades do sistema foram definidas para atender ao escopo geral da gestão dos cães, desde um cadastro atualizado com informações relevantes sobre os mesmo até campanhas de ajuda financeira, para tratar cuidados com cirurgias, veterinários e dentre outras necessidades. A seguir, é detalhado cada um desses requisitos, especificando suas descrições, critérios de aceite e os atores envolvidos.

3.1.1 Requisitos Funcionais

Nesta seção, serão apresentados os requisitos funcionais do sistema, que foram levantados a partir de uma análise sobre o que foi conversado durante a entrevista semiestruturada.

3.1.1.1 RF01 - Permitir que um usuário crie/edite o perfil de um cão

- **Prioridade:** Alta
- **Descrição:**
 - O sistema deve permitir que qualquer usuário logado possa criar e editar o perfil de um cão.
- **Crerios de Aceite:**
 1. O perfil de um cão deve ser cadastrado com sucesso no banco de dados.
 2. As informações do perfil de um cão existente devem poder ser editadas e salvas.

- **Atores Envolvidos:** Usuário, Sistema.

3.1.1.2 RF02 - *O perfil de um cão deve conter diversas informações sobre o mesmo*

- **Prioridade:** Alta

- **Descrição:**

- O sistema deve permitir que um perfil de cão armazene nome, fotos, descrição (física e personalidade), histórico de saúde, *status* atual (Ex: para adoção, em lar temporário, desaparecido, adotado), e sua última localização/matilha conhecida.

- **Critérios de Aceite:**

1. O perfil do cão deve permitir o cadastro de uma ou mais fotos (até 4).
2. O perfil do cão deve permitir o cadastro de seu nome, descrição, histórico e *status*.
3. O perfil do cão deve exibir a última localização conhecida, atualizada via avis-tamentos.

- **Atores Envolvidos:** Usuário, Sistema.

3.1.1.3 RF03 - *Permitir um histórico de eventos para cada cão*

- **Prioridade:** Alta

- **Descrição:**

- O sistema deve permitir que usuários registrem um histórico de eventos para um cão, escolhendo entre tipos pré-definidos como: Avistamento, Saúde, Transporte e Alerta (Desaparecido).

- **Critérios de Aceite:**

1. O sistema deve permitir o cadastro de eventos dos tipos Avistamento, Saúde, Transporte e Alerta.
2. A tela de perfil do cão deve exibir o histórico de eventos em ordem cronoló-gica.
3. O sistema deve permitir a consulta do histórico, com filtros por tipo de evento e data.

- **Atores Envolvidos:** Usuário, Sistema.

3.1.1.4 RF04 - *Exibir um mapa interativo com avistamentos*

- **Prioridade:** Alta

- **Descrição:**

- O sistema deve fornecer um mapa interativo que exiba marcadores ("pontos de avistamento") com a localização dos avistamentos recentes.

- **Critérios de Aceite:**

1. O mapa deve exibir apenas o último avistamento reportado para cada cão para não poluir a visualização.
2. Avistamentos anteriores podem ser consultados no histórico de eventos do cão (RF03).
3. Avistamentos de cães com *status* "desaparecido" devem possuir um marcador com destaque visual (cor vermelha).

- **Atores Envolvidos:** Usuário, Sistema.

3.1.1.5 RF05 - Permitir que um usuário reporte um avistamento

- **Prioridade:** Alta

- **Descrição:**

- O sistema deve permitir que qualquer usuário reporte ter visto um cão, iniciando o fluxo pela câmera do dispositivo para anexar uma foto, registrando automaticamente a localização e o horário.

- **Critérios de Aceite:**

1. O fluxo deve iniciar com a abertura da câmera para tirar uma foto como comprovação.
2. Após a foto, o sistema deve registrar a localização e o horário atuais.
3. O registro do avistamento deve ser disparado como um evento do tipo "Avistamento" no histórico do cão, se ele for identificado.

- **Atores Envolvidos:** Usuário, Sistema.

3.1.1.6 RF06 - Usuários podem marcar um cão como "Desaparecido"

- **Prioridade:** Alta

- **Descrição:**

- O sistema deve permitir que um usuário marque um cão como "Desaparecido" através do registro de um evento de "Alerta", o que deve mudar o *status* do cão e notificar os seguidores.

- **Critérios de Aceite:**

1. Ao registrar um evento do tipo "Alerta/DESAPARECIDO", o *status* do cão deve ser atualizado para "desaparecido".
2. O perfil do cão deve exibir um selo visual indicando o *status* "DESAPARECIDO".
3. Uma notificação *push* deve ser enviada para todos os usuários que seguem o cão.
4. Os marcadores de avistamento deste cão no mapa devem mudar para a cor vermelha.

- **Atores Envolvidos:** Usuário, Sistema.

3.1.1.7 RF07 - Permitir a criação de campanhas de arrecadação

- **Prioridade:** Alta

- **Descrição:**

- O sistema deve permitir a criação de campanhas de arrecadação (pedidos de ajuda financeiros) para necessidades específicas de um cão, com meta, prazo e integração com um *gateway* de pagamento para doações.

- **Critérios de Aceite:**

1. O criador da campanha deve poder definir uma meta, prazo (30 dias por padrão), e a chave PIX para saque.
2. Doadores devem poder doar através do *app*, gerando uma cobrança PIX via *gateway* de pagamento.
3. O sistema deve registrar todas as doações (quem doou, quanto, quando) para fins de transparência.
4. O valor total arrecadado deve ser atualizado em tempo real na tela da campanha.
5. O criador da campanha deve poder simular o saque do dinheiro arrecadado.

- **Atores Envolvidos:** Usuário (Criador), Usuário (Doador), Sistema, *Gateway* de Pagamento.

3.1.1.8 RF08 - Criadores de campanhas devem prestar contas

- **Prioridade:** Alta

- **Descrição:**

- O sistema deve prover uma funcionalidade para que os criadores de campanhas financeiras possam prestar contas de como o dinheiro arrecadado foi utilizado, anexando fotos e descrições.

- **Critérios de Aceite:**

1. O criador deve poder adicionar itens de prestação de contas com foto e descrição.
2. A seção de prestação de contas deve ser visível para qualquer usuário na página da campanha.
3. Doadores da campanha devem ser notificados quando uma nova prestação de contas for adicionada.

- **Atores Envolvidos:** Usuário (Criador), Usuário (Doador), Sistema.

3.1.1.9 RF09 - Usuários podem seguir cães

- **Prioridade:** Média
- **Descrição:**
 - O sistema deve permitir que usuários sigam cães de seu interesse para receberem notificações prioritárias sobre eles. (Anteriormente chamado de "Apadrinhamento").
- **Critérios de Aceite:**
 1. O usuário que segue um cão deve receber notificações de qualquer evento novo (Avistamento, Alerta, Saúde, etc.) relacionado ao cão.
- **Atores Envolvidos:** Usuário, Sistema.

3.1.1.10 RF10 - Criação de "Pedidos de Ajuda" não financeiros

- **Prioridade:** Média
- **Descrição:**
 - O sistema deve permitir a criação de pedidos de ajuda para necessidades não financeiras, como transporte, companhia, etc.
- **Critérios de Aceite:**
 1. O pedido de ajuda deve conter Título, Descrição e um Cão anexado (opcional).
 2. Os pedidos de ajuda devem ser listados em uma seção de fácil acesso no *app*.
 3. O criador do pedido deve poder marcá-lo como "Resolvido".
- **Atores Envolvidos:** Usuário, Sistema.

3.1.1.11 RF11 - Chat entre usuários

- **Prioridade:** Média
- **Descrição:**
 - O sistema deve fornecer um sistema de *chat peer-to-peer* para facilitar a comunicação entre o criador de um pedido de ajuda e um voluntário/doador.
- **Critérios de Aceite:**
 1. O *chat* só deve ser criado a partir da interação com um pedido de ajuda.
 2. Para pedidos não financeiros, o criador deve aceitar uma "solicitação de ajuda" para iniciar o *chat*.
 3. Para pedidos financeiros, um *chat* é disponibilizado após a doação.
 4. O sistema deve possuir uma tela que lista todas as conversas do usuário.
 5. Novas mensagens devem gerar notificações *push* para o destinatário.
- **Atores Envolvidos:** Usuário, Sistema.

3.1.2 Requisitos Não Funcionais

Nesta seção, são discutidos os requisitos não funcionais do sistema Cãomunidade, que especificam critérios de qualidade e atributos que o aplicativo deve possuir.

3.1.2.1 RNF01 - Usabilidade

- **Categoria:** Usabilidade
- **Requisito:** A interface do aplicativo deve ser intuitiva.
- **Descrição:** O fluxo para realizar ações básicas no *app* deve ser rápido e claro, mesmo para usuários não-técnicos.
- **Critérios de Aceite:**
 1. As 3 principais ações (Reportar Avistamento, Pedir Ajuda, Ver Cães) devem estar acessíveis a partir da tela inicial.
 2. Um novo usuário deve conseguir completar o fluxo de "Cadastrar um Avistamento" em menos de 60 segundos.

3.1.2.2 RNF02 - Acessibilidade

- **Categoria:** Acessibilidade
- **Requisito:** O *design* do aplicativo deve ser acessível.
- **Descrição:** O aplicativo deve seguir boas práticas de acessibilidade para garantir o uso por um público mais amplo.
- **Critérios de Aceite:**
 1. Todas as imagens funcionais (ícones) devem ter uma descrição textual alternativa para leitores de tela.
 2. O tamanho da fonte do aplicativo deve respeitar as configurações de acessibilidade do sistema operacional.

3.1.2.3 RNF03 - Confiabilidade (Notificações)

- **Categoria:** Confiabilidade
- **Requisito:** O sistema de notificações de emergência deve ser altamente confiável.
- **Descrição:** Notificações críticas, como "Cão Desaparecido", devem ser entregues de forma rápida e garantida.
- **Critérios de Aceite:**
 1. O sistema deve garantir a tentativa de entrega de 99.9% das notificações de "Alerta".
 2. O tempo entre o evento ser gerado e a notificação ser enviada não deve exceder 30 segundos.

3.1.2.4 RNF04 - Desempenho

- **Categoria:** Desempenho
- **Requisito:** O aplicativo deve ser leve e responsivo.
- **Descrição:** O aplicativo deve ter tempos de carregamento curtos e interações fluidas.
- **Critérios de Aceite:**
 1. O tempo de carregamento inicial do *app* deve ser inferior a 5 segundos.
 2. A transição entre as telas principais deve ocorrer em menos de 1 segundo.
 3. O mapa interativo deve carregar os marcadores em menos de 5 segundos.

3.1.2.5 RNF05 - Transparência

- **Categoria:** Transparência
- **Requisito:** O sistema deve prover mecanismos claros de transparência financeira.
- **Descrição:** O sistema deve fortalecer a confiança dos usuários quanto à transparência financeira das campanhas.
- **Critérios de Aceite:**
 1. Toda campanha de arrecadação deve ter uma seção pública de "Prestações de Contas".
 2. A lista de doadores (respeitando o anonimato) e os valores doados devem ser visíveis na página da campanha.
 3. O sistema deve registrar o histórico de saques realizados pelo criador da campanha.

3.2 Diagrama de Casos de Uso

Nessa sessão será apresentado o diagrama de casos de uso do Cãomunidade, representado na Figura 3.1. A figura possui os casos de uso **Cadastrar Usuário**, **Cadastrar Cão**, **Seguir Cão**, **Cadastrar Eventos**, **Cadastrar Avistamentos** e **Cadastrar Pedidos de Ajuda**, que inclui os casos de uso **Pagar**, **Sacar** e **Prestar Contas** e **Trocar Mensagens**.

Na Figura 3.1 o ator **Usuário** pode realizar diversas ações no sistema.

3.2.1 Casos de Uso

Os detalhes dos casos de uso apresentados na Figura 3.1 são os seguintes:

- **Cadastrar Usuário:** Este caso de uso permite que um usuário possa se cadastrar no sistema, informando nome, *e-mail* e senha. Ele implementa o requisito funcional RF13.
- **Cadastrar Cão:** Este caso de uso permite que um usuário possa cadastrar o perfil de um cão, informando nome, fotos, histórico médico e *tags*. Ele implementa os requisitos funcionais RF01 e RF02.

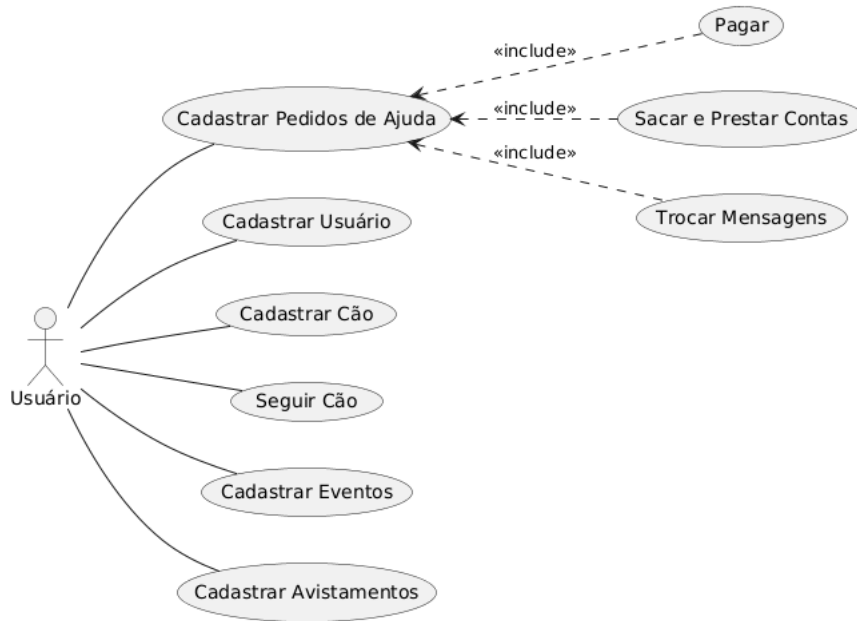


Figura 3.1: Diagrama de Casos de Uso

- **Seguir Cão:** Este caso de uso permite que um usuário possa seguir um cão, recebendo notificações sobre o mesmo, referentes á novos pedidos de ajuda, doações e eventos. Ele implementa o requisito funcional RF09.
- **Cadastrar Eventos:** Este caso de uso permite que um usuário possa cadastrar eventos para um cão, que são definidos por administradores do sistema, como o evento do tipo “Desapareceu”. Ele implementa os requisitos funcionais RF03 e RF06.
- **Cadastrar Avistamentos:** Este caso de uso permite que um usuário possa cadastrar avistamentos para um cão, marcando em um mapa a partir de uma foto tirada pelo celular aonde um cão foi visto pela última vez. Ele implementa os requisitos funcionais RF04 e RF05.
- **Cadastrar Pedidos de Ajuda:** Este caso de uso permite que um usuário possa cadastrar pedidos de ajuda financeiros e não-financeiros para um cão. Ele implementa os requisitos funcionais RF10 e RF07.
- **Pagar:** Este caso de uso permite que um usuário possa doar para pedidos de ajuda financeiros. Ele implementa um dos critérios de aceite do requisito funcional RF07.
- **Sacar e Prestar Contas:** Este caso de uso permite que um usuário que criou um pedido de ajuda possa sacar o dinheiro arrecadado e prestar contas. Ele implementa o requisito funcional RF08.
- **Trocar Mensagens:** Este caso de uso permite que usuários do sistema possam trocar mensagens entre si. Ele implementa o requisito funcional RF11.

3.3 Diagrama de Atividades

Nesta seção, serão exibidos alguns diagramas de atividades do aplicativo, para um melhor entendimento de como são realizadas as transações financeiras dentro do mesmo.

Estes diagramas são **Criar Pedido de Ajuda Financeiro** na Figura 3.2, **Realizar Doação via PIX** na Figura 3.3 e **Realizar Saque via PIX** na Figura 3.4. Os diagramas detalham os fluxos de interação para as funcionalidades críticas relacionadas aos pedidos de ajuda financeiros no sistema, ilustrando o passo a passo das ações realizadas pelo o **Usuário** e pelo o **Sistema**, incluindo a integração com o *gateway* de pagamento **AbacatePay**.

3.3.1 Criar Pedido de Ajuda Financeiro

O diagrama na Figura 3.2 demonstra o fluxo simplificado para a criação de um pedido de ajuda financeiro por um usuário do aplicativo.

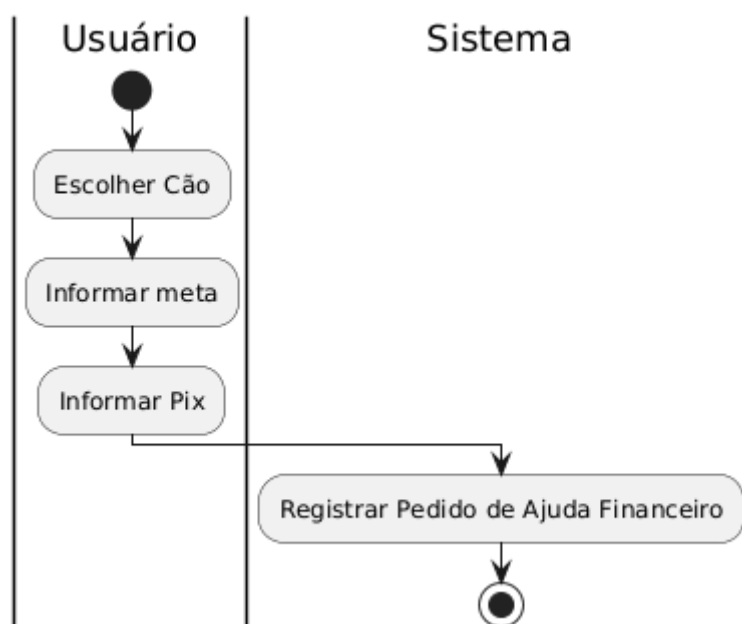


Figura 3.2: Diagrama de Atividade - Criar Pedido de Ajuda Financeiro

O processo inicia com o **Usuário** (criador do pedido) interagindo com o sistema. Primeiramente, ele seleciona o cão ao qual o pedido de ajuda é direcionado. Em seguida, informa a meta financeira do pedido, o tipo da chave PIX e a chave em si que será usada para o saque dos fundos arrecadados. Com as informações financeiras necessárias preenchidas, o **Sistema** então registra o novo pedido de ajuda como um pedido de ajuda financeiro no banco de dados (Firestore), marcando-o como ativo, notificando os seguidores do cão do pedido e iniciando o seu valor arrecadado igual a zero, finalizando o fluxo de criação de um pedido de ajuda financeiro.

3.3.2 Realizar Doação via PIX

O fluxo de uma doação, representado na Figura 3.3, envolve a interação entre o usuário (doador), o sistema Cãomunidade e o *gateway* de pagamento AbacatePay.

O **Usuário** (doador) inicia o processo selecionando um pedido de ajuda financeiro ativo, pressionando o botão de doar e informando o valor que deseja doar. O **Sistema** recebe esse valor e solicita (via API) a geração de uma cobrança PIX ao *gateway* de pagamento AbacatePay, enviando o valor da doação. O AbacatePay gera a cobrança e retorna os dados do PIX para o Sistema, que são um código "copia e cola", que o usuário pode copiar e colar em seu aplicativo de banco de preferência para realizar o PIX, e um *QR Code*,

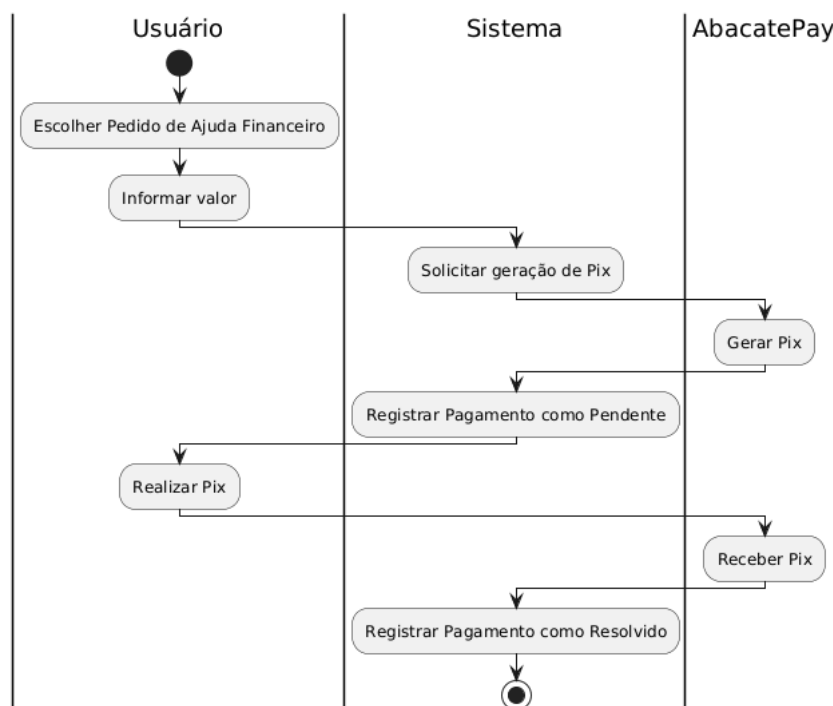


Figura 3.3: Diagrama de Atividade - Realizar Doação via PIX

que ele pode usar para que outra pessoa escaneie e realize a doação. Neste momento, o Sistema, ao receber os dados do PIX, renderiza o *QR Code* em tela e devolve a chave "copia e cola" para que o usuário possa pagar no *app* de um banco de sua preferência. Ao mesmo tempo, o Sistema registra a doação no banco de dados com o *status* PENDENTE, anexando o ID da transação do *gateway* (para futuramente alterar *status* da doação). O Usuário, então, realiza o pagamento do PIX. O AbacatePay recebe a confirmação do pagamento e notifica o Sistema (através de um *webhook*). Ao receber a confirmação via *webhook*, o Sistema atualiza o *status* da doação para PAGO no banco de dados e incrementa o *valor_arrecadado* do pedido de ajuda correspondente, concluindo o fluxo de doação.

3.3.3 Realizar Saque via PIX

A Figura 3.4 ilustra o processo de saque dos valores arrecadados por parte do criador do pedido de ajuda financeiro. O processo de saque é simulado, devido ao sistema ser desenvolvido usando somente o ambiente de testes do AbacatePay, e não sendo possível realizar transações financeiras reais em decorrência disso.

O **Usuário** (criador do pedido) inicia o fluxo escolhendo o pedido de ajuda financeiro do qual deseja sacar os fundos arrecadados e informa o valor do saque. O **Sistema** primeiro verifica se há saldo disponível (calculado como *valor_arrecadado* - *valor_sacado*) e se ele é maior ou igual ao valor solicitado. Se o saldo for insuficiente, o fluxo é encerrado. Caso contrário, o Sistema solicita o saque (via API) ao AbacatePay, enviando o valor, o tipo da chave PIX e a chave PIX em si cadastrada pelo criador. O AbacatePay processa a transferência PIX, que por sua vez é uma simulação de uma transferência real, já que o sistema utiliza somente o ambiente de testes do AbacatePay. Após a confirmação (simulada) do *gateway*, o Sistema atualiza o campo *valor_sacado* no documento do pedido de ajuda no Firestore, finalizando o processo.

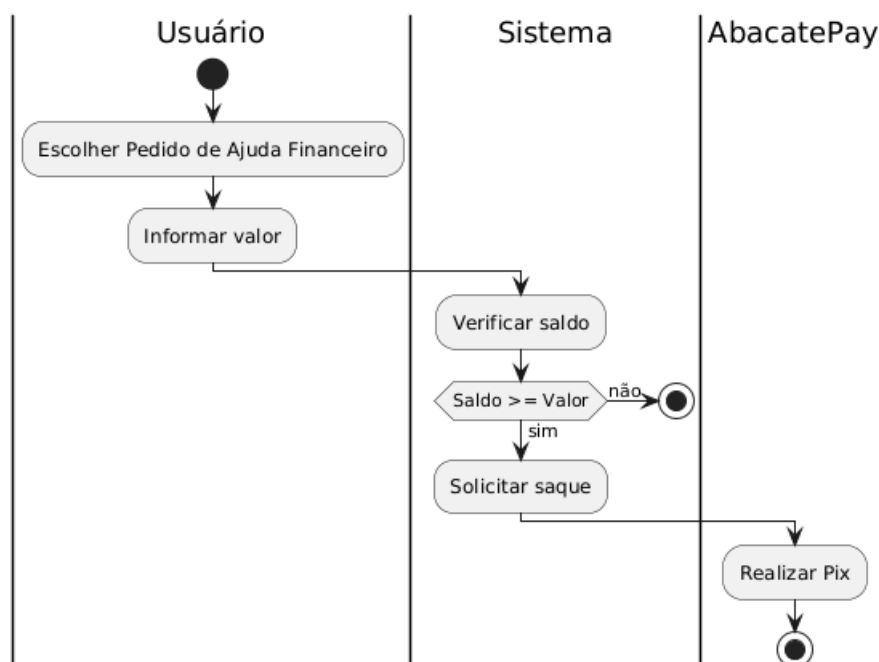


Figura 3.4: Diagrama de Atividade - Realizar Saque via PIX

3.4 Prototipação de Tela

Nesta seção, será abordada a prototipação de telas do sistema, que foi feita para se obter uma base visual para o desenvolvimento do *front-end*, com foco em garantir uma experiência de usuário agradável e intuitiva. Em seguida, serão apresentados cinco protótipos de tela dentre vinte e quatro protótipos, sendo eles a **Tela de Avistamentos**, **Tela de Cães**, **Tela de Eventos**, **Tela de Pedidos de Ajuda** e **Tela de Pedido de Ajuda**. Estes protótipos foram escolhidos para dar um panorama sobre as principais funcionalidades do sistema, revelando como havia sido concebido as ideias iniciais quanto á fluxos de tela, paleta de cores e poscionamento de campos.

3.4.1 Tela de Avistamentos

A Figura 3.5 ilustra o protótipo da tela de **Avistamentos**, responsável por mostrar em tela um mapa com todos os avistamentos de todos os cães do sistema realizados nos últimos dois meses, disponibilizando somente um filtro por cães desaparecidos. Posteriormente, durante o desenvolvimento do sistema, foi incluído mais filtros, possibilitando filtrar os avistamentos por cães selecionados, *tags* dos cães, gênero e se o cão está desaparecido.

A Figura 3.6 ilustra o protótipo da tela de **Cães**, que é responsável por listar os cães cadastrados no sistema, possibilitando o acesso ao perfil de algum cão específico. A tela de Cães também conta com filtros por nome de cão, *tags*, gênero e se está desaparecido ou não.

A Figura 3.7 ilustra o protótipo da tela de **Eventos**, que é responsável por listar os eventos de um cão, que podem ser cadastrados por qualquer usuário do sistema. Os eventos de um cão podem ser de vários tipos, sendo que os tipos de eventos em si só podem ser cadastrados por usuários que são administradores do sistema. Na Figura 3.7 é possível ver exemplos de eventos de avistamento, cão desaparecido, saúde e lar temporário.



Figura 3.5: Protótipo - Tela de Avistamentos



Figura 3.6: Protótipo - Tela de Cães

A Figura 3.8 ilustra o protótipo da tela de **Pedidos de Ajuda**, que é responsável por listar os pedidos de ajuda do sistema, ambos financeiros e não-financeiros. Na Figura 3.8 é possível ver destaques de cor diferentes para diferentes tipos de pedido de ajuda, com os financeiros tendo uma cor verde-claro e os não-financeiros tendo uma cor amarela, e os pedidos resolvidos tendo uma cor verde.

A Figura 3.9 ilustra o protótipo da tela de visualização de um **Pedido de Ajuda financeiro**, a partir da visão de um usuário que não é o criador do pedido. Na Figura 3.9 é possível ver dois botões: o botão para realizar uma doação (doar) e o botão para visualizar a prestação de contas, que leva para a tela de listagem das prestações de contas.

Nome do Cão

Tipo do evento:

Data: De Até

{Nome do Cão} foi avistado
Dia xx/xx/xxxx às xx:xx

{Nome do Cão} foi marcado
como desaparecido
Dia xx/xx/xxxx às xx:xx

{Nome do Cão} começou
medicação nova (data e etc.)

{Nome do cão} está em lar
temporário X (data e etc.)

Figura 3.7: Protótipo - Tela de Eventos

Pedidos de ajuda

Descrição do pedido financeiro
Meta: R\$xx.xxx,xx Arrecadado: R\$xxxx

Descrição do pedido
Data:xx/xx/xxxx xx:xx Local: etc.

Descrição do pedido financeiro
Pedido resolvido

Descrição do pedido
Data:xx/xx/xxxx xx:xx Local: etc.

Descrição do pedido
Data:xx/xx/xxxx xx:xx Local: etc.

Figura 3.8: Protótipo - Tela de Pedidos de ajuda

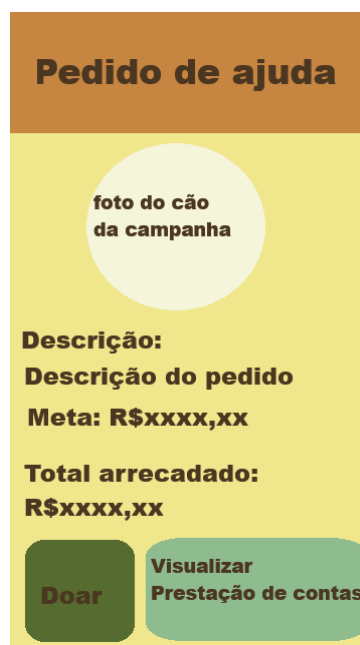


Figura 3.9: Protótipo - Tela de Pedidos de ajuda

4 ARQUITETURA E PROJETO

4.1 Diagrama de Arquitetura

O Diagrama de Arquitetura apresentado na Figura 4.1 demonstra a relação entre as diferentes camadas e módulos do sistema.

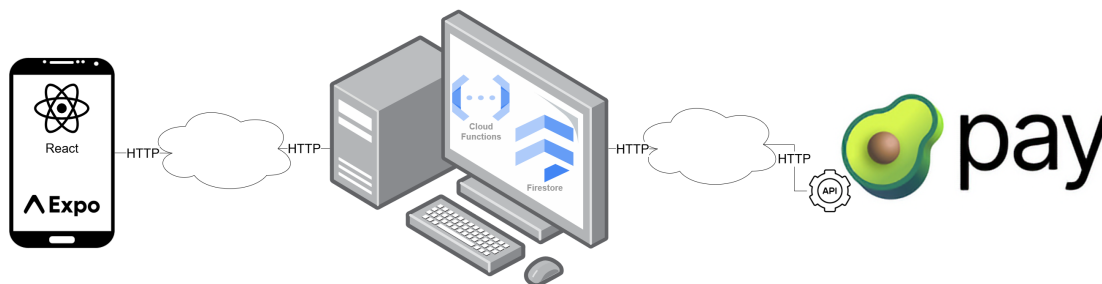


Figura 4.1: Diagrama de Arquitetura

No *front-end*, o React Native é o responsável pela interface do usuário, partindo do arquivo `App.js` para conectar o `navigator` as diversas telas do sistema.

O Firebase é responsável pelo banco de dados NoSQL em tempo real (Firestore) e pelas funções em nuvem (*Cloud Functions*), que, por sua vez, são usadas para autenticar o usuário, tratar o envio de notificações de forma segura e se comunicar com a API do AbacatePay.

O AbacatePay é um *gateway* de pagamento utilizado para processar as transações PIX realizadas dentro do sistema, provendo um meio de receber e enviar dinheiro para os pedidos de ajuda financeiros.

O Expo é utilizado para testar e construir o aplicativo, integrar funcionalidades de notificações *push* (Expo Notifications) e câmera (Expo Camera).

Esse modelo de arquitetura garante escalabilidade e eficiência, provendo um sistema com separações de responsabilidades bem definidas entre as camadas.

4.2 Diagrama de Classes

Nesta seção será apresentado na Figura 4.2 o diagrama das classes definidas no *front-end* do aplicativo e seus relacionamentos com o *backend* do Firebase e a API de notificações *push* do Expo.

A classe **abacatePayService** atua como uma interface cliente para as *Cloud Functions* do Firebase, abstraindo a comunicação entre elas e o *gateway* de pagamento do AbacatePay.

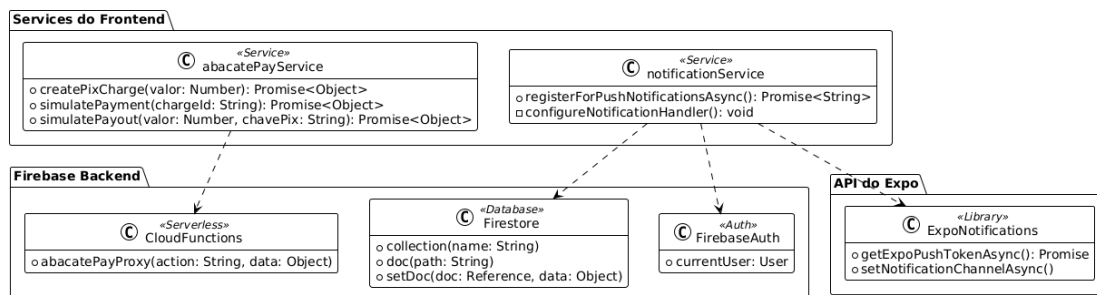


Figura 4.2: Diagrama de Classes

A classe **notificationService** gerencia o ciclo de vida das notificações *push*, interagindo com a biblioteca do Expo para obter o *token* do dispositivo e, subsequentemente, persistindo-o no documento do usuário no Firestore, garantindo o envio de notificações direcionadas no *backend*

4.3 Diagrama de Documentos

Nesta seção será apresentado na Figura 4.3 o Diagrama de Documentos, que juntos formam o banco de dados NoSQL do Firestore, utilizado no desenvolvimento do aplicativo. O Firestore utiliza uma estrutura de banco de dados baseada em **documentos**, que possuem **coleções**, que são grupos de documentos que organizam os dados do banco de dados. Cada coleção pode possuir uma **subcoleção**, que são coleções filhas de outras coleções.

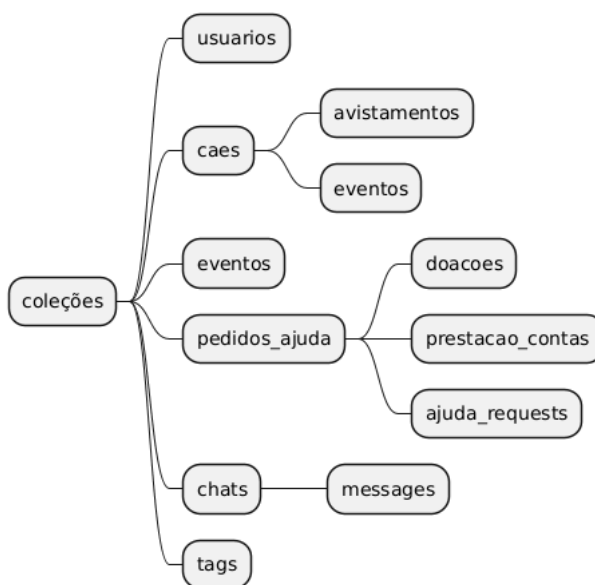


Figura 4.3: Diagrama de Documentos

As coleções utilizadas para armazenar os dados do aplicativo são descritas a seguir:

- **users:** Esta coleção, apresentada na Figura 4.4, é responsável por armazenar os dados do usuário (nome, email e eh_admin), seu pushToken, que é utilizado para o envio de notificações e também quais cães ele segue (caes_seguindo_ids).



Figura 4.4: Coleção users

- **caes:** Esta coleção, apresentada na Figura 4.5, é responsável por armazenar os dados básicos dos cães (nome, descricao, genero, porte, historico_medico), como também as suas fotos (foto_principal_url e fotos_urls), suas tags, palavras-chave do seu nome (keywords_nome), que são utilizadas para facilitar a busca pelo nome de um cão, seu status, o id do usuário criador (id_usuario_criador), a sua data de cadastro (data_cadastro) e a flag ehDesconhecido, que serve apenas para diferenciar o cadastro do cão “desconhecido” do restante dos cães. Também dentro desta coleção temos duas subcoleções, que são **avistamentos** e **eventos**.
- **avistamentos (subcoleção):** Esta subcoleção, apresentada na Figura 4.5, é responsável por armazenar os avistamentos de um cão. Contém sua data e hora de registro (data_hora), quem cadastrou (id_usuario), a localização em latitude e longitude, armazenados usando o *GeoPoint* do Firestore (localizacao), a foto do avistamento (foto_url), o nome e o id do cão (caoNome e caoId).
- **eventos (subcoleção):** Esta subcoleção, apresentada na Figura 4.5, é responsável por armazenar os eventos de um cão. Contém a descrição do evento(descricao), a cor do evento (cor) que é exibida na listagem, a data que o evento foi cadastrado (data) e o local do evento (local), caso seja um evento de local obrigatório.

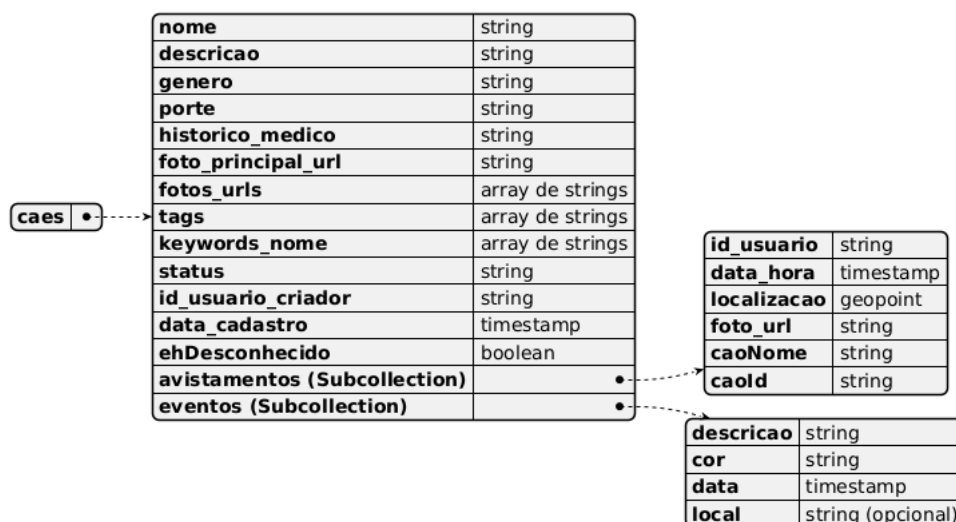


Figura 4.5: Coleção caes

- **eventos :** Esta coleção, apresentada na Figura 4.6, é responsável por armazenar os tipos de eventos de um cão. Contém a descrição, a cor e se o local é obrigatório no momento do cadastro (descricao, cor e local_obrigatorio)



Figura 4.6: Coleção eventos

- **pedidos_ajuda:** Esta coleção, apresentada na Figura 4.7, é responsável por armazenar os pedidos de ajuda, sejam eles financeiros ou não. Contém os dados essenciais do pedido como `titulo`, `descricao`, quem o criou (`id_usuario_criador`), informações básicas do cão associado (`caoInfo`), as datas de criação e expiração (`data_criacao`, `data_expiracao`), um indicador se o pedido foi resolvido (`resolvido`), e um *boolean* (`ehFinanceiro`) para diferenciar os tipos de pedido. Para pedidos financeiros, armazena a meta, o `valor_arrecadado`, o `valor_sacado` e os dados do PIX para saque (`pix`). Guarda também uma lista dos IDs dos usuários que se ofereceram para ajudar (`ajudantes_ids`) e uma *flag* opcional (`metaNotificada`) para controle de notificações. Possui três subcoleções: **doacoes**, **prestacao_contas** e **ajuda_requests**.
- **doacoes (subcoleção):** Esta subcoleção, apresentada na Figura 4.7, é responsável por armazenar cada doação realizada. Contém o ID do doador (`id_usuario_doador`), o valor doado, o *status* do pagamento (Ex: 'PAGO'), a data da doação (`data_criacao`) e o ID da transação no *gateway* de pagamento (`id_transacao_gateway`).
- **prestacao_contas (subcoleção):** Esta subcoleção, apresentada na Figura 4.7, é responsável por armazenar os itens de prestação de contas adicionados pelo criador. Cada item contém a URL da foto comprobatória (`foto_url`), uma *descricao* textual e a data de criação do item (`data_criacao`).
- **ajuda_requests (subcoleção):** Esta subcoleção, apresentada na Figura 4.7, é responsável por armazenar as solicitações de usuários que desejam ajudar. Cada documento (com ID igual ao do ajudante) contém o nome do ajudante (`ajudanteNome`), o *status* da solicitação (Ex: 'pendente') e a data da solicitação (`requestedAt`).
- **chats:** Esta coleção, apresentada na Figura 4.8, é responsável por armazenar as conversas entre o criador de um pedido de ajuda e um ajudante/doador. Contém o ID e título do pedido original (`pedidoId`, `pedidoTitulo`), um *array* com os IDs dos dois participantes (`participants`), um mapa com os nomes dos participantes (`participantInfo`), os dados da última mensagem enviada (`lastMessage`), a data de criação do *chat* (`createdAt`) e um objeto para indicar quem está digitando (`typing`). Possui uma subcoleção **messages**.
- **messages (subcoleção):** Esta subcoleção, apresentada na Figura 4.8, é responsável por armazenar cada mensagem individual da conversa. Contém o texto da mensagem (`text`), a data de criação (`createdAt`) e o ID do usuário que enviou (`userId`).
- **tags:** Esta coleção, apresentada na Figura 4.9, é utilizada para otimizar a busca e sugestão de *tags* no cadastro de cães. Cada documento possui como ID a própria *tag* em minúsculas (Ex: 'docil') e contém apenas um campo `tagName` com o valor da *tag*, para facilitar a consulta.

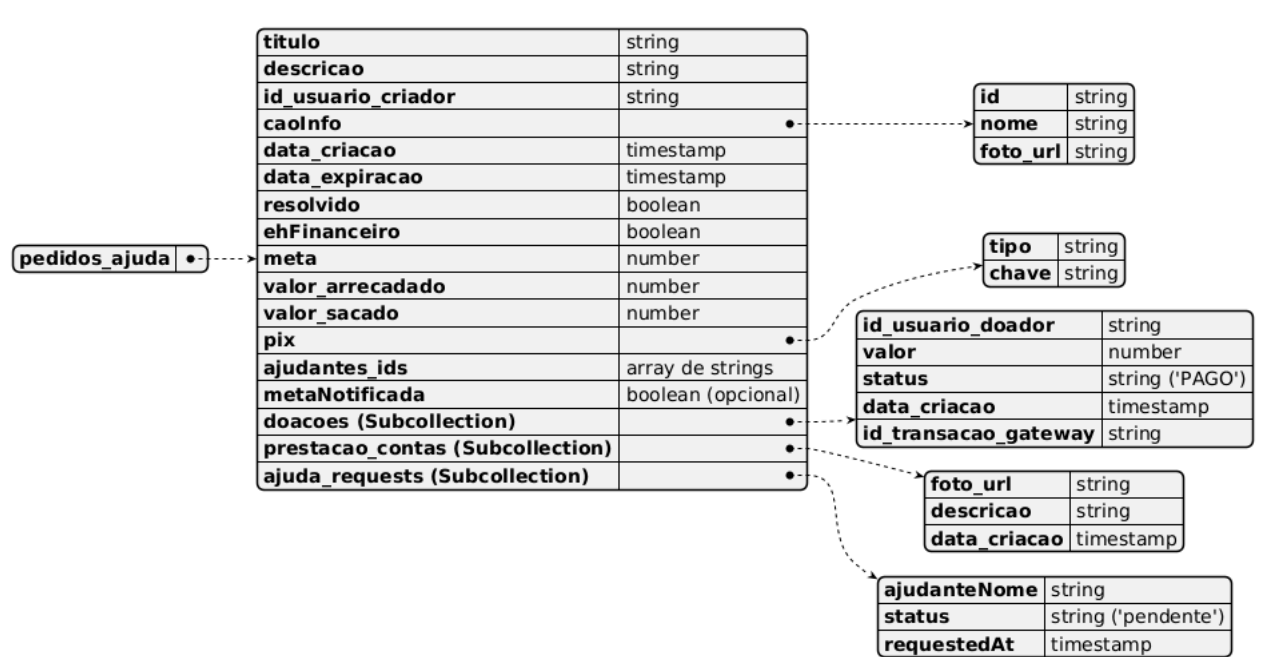


Figura 4.7: Coleção pedidos_ajuda

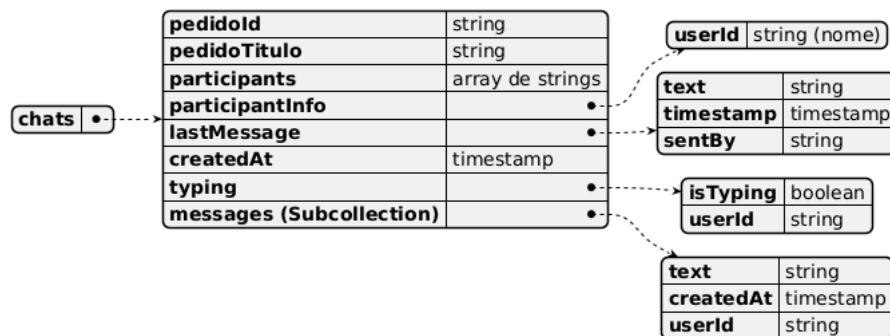


Figura 4.8: Coleção chats



Figura 4.9: Coleção tags

5 IMPLEMENTAÇÃO

5.1 Ferramentas

A seleção de ferramentas adequadas foi uma etapa crucial para o desenvolvimento do sistema, visando garantir uma solução técnica viável, de fácil manutenção e capaz de escalar em um nível maior. Nesta seção serão detalhadas as tecnologias que foram utilizadas no desenvolvimento do sistema, bem como o porquê de elas terem sido escolhidas.

5.1.1 React Native

O React Native é um *framework* de código aberto, mantido pela Meta, que permite o desenvolvimento de aplicativos móveis nativamente compatíveis com sistemas Android e iOS, utilizando o Javascript e a biblioteca React (Meta Platforms, Inc., 2025a). Sua principal vantagem reside na capacidade de um único algoritmo ser compatível dentre várias plataformas, acelerando o desenvolvimento e facilitando a compatibilidade entre sistemas operacionais diferentes.

Criar dois aplicativos nativos separados seria inviável para o escopo do sistema, e é justamente este o problema que o React Native soluciona, possibilitando um reuso de boa parte do código do sistema entre as principais plataformas de dispositivos móveis, que são o **Android** e o **iOS**.

Diferente de abordagens híbridas mais antigas que utilizam *WebViews*, que são páginas de navegadores incorporadas dentro de um aplicativo, o React Native utiliza uma ponte assíncrona para se comunicar com os componentes de interface nativos de cada sistema operacional (Meta Platforms, Inc., 2025b). Isso resulta em uma performance muito superior e uma compatibilidade quase instantânea com a maioria dos sistemas operacionais de dispositivos móveis e navegadores *web*.

A escolha se deu também pela vasta comunidade e ecossistema de bibliotecas, que facilitam a implementação de funcionalidades diversas.

5.1.1.1 React Navigation

O React Native Navigation é uma biblioteca nativa do React Native que permite a navegação entre diferentes telas do sistema de forma ágil e eficiente. Esta biblioteca tem suporte a diferentes tipos de navegação, no sistema foi implementado o *Stack Navigator*, que seria uma estrutura de pilha (React Navigation Contributors, 2025).

5.1.1.2 React Native Maps

O React Native Maps foi utilizado para fornecer um dos componentes fundamentais do sistema, que é o mapa de avistamentos. O Maps permite a implementação de mapas

interativos, com marcadores (*pins*) com as localidades reportadas pelos usuários (React Native Community, 2025).

5.1.2 Expo

O Expo é ao mesmo tempo uma plataforma e um conjunto de ferramentas construídas sobre o React Native, projetadas para simplificar o desenvolvimento, os testes, a construção (*build*) e a implantação de aplicativos universais (iOS, Android e *Web*) (Expo, 2025a). A escolha do Expo para compor as ferramentas do sistema se deve também às suas bibliotecas nativas que dão suporte a diversos recursos dos dispositivos móveis, como a localização (usando o `expo-location`), a câmera (usando `expo-camera`) e o acesso a galeria de imagens (usando o `expo-image-picker`).

5.1.2.1 Expo Notifications

O Expo Notifications é a API do Expo usada para o envio de notificações do sistema, permitindo notificar diversos dispositivos ao mesmo. Por sua fácil implementação e agilidade no envio, a API Expo Notifications se provou uma escolha fundamental para o desenvolvimento do sistema (Expo, 2025b).

5.1.3 Firebase

O Firebase é uma plataforma de desenvolvimento do Google que funciona como um provedor de serviços de *backend* para diversos aplicativos ao redor do mundo (Google LLC, 2025a). Sua performance e facilidade de implementação se tornaram qualidades essenciais para garantir robustez ao *backend* do sistema, provendo diversas funcionalidades cruciais para cuidar de toda a lógica de regras de negócio e de armazenamento de dados do sistema. A suíte integrada do Firebase permitiu centralizar todas as funcionalidades de *backend* em um único ecossistema.

5.1.3.1 Firebase Authentication

O Firebase Authentication é usado para gerenciar a autenticação de *login* dos usuários do sistema, e também providencia um método fácil de cadastro, necessitando somente do *E-mail* e uma senha para que o usuário seja cadastrado no sistema (Google LLC, 2025b). A justificativa para a sua escolha foi a segurança: o Authentication lida nativamente com o armazenamento e *hashing* de senhas, gerenciamento de sessão e redefinição de senha, eliminando a necessidade de implementar essa lógica manualmente no *backend* do sistema.

5.1.3.2 Firebase Firestore

O Firestore é o banco de dados NoSQL em nuvem do Firebase (Google LLC, 2025c), que possui uma capacidade de sincronização de dados em tempo real (Google LLC, 2025d), permitindo a funcionalidade de *chat*, atualização instantânea de valores arrecadados pelas campanhas e de avistamentos no mapa, sem necessidade de recarregar o aplicativo. A decisão de escolher um banco de dados NoSQL ao invés de um relacional se deve ao enorme potencial de escalabilidade e evolução que um esquema flexível como o do Firestore providencia, tornando um eventual aumento de escopo do sistema possível.

O Firestore também permite a definição de Regras (Google LLC, 2025e), que são medidas de segurança bastante flexíveis referente ao banco de dados, com um funcionamento semelhante às *roles* de outros bancos de dados, mas disponibilizando uma gama maior de

possibilidades que uma regra possa se aplicar, desde todas as gravações de um banco de dados até operações de um documento específico.

5.1.3.3 *Firebase Functions*

O Functions é o *backend serverless* do Firebase, usado para processar chamadas de API (Google LLC, 2025f). Escritas em Typescript, as Functions permitem o cadastro de gatilhos para o banco de dados e um *proxy* seguro para requisições, armazenando as variáveis de ambiente, que são definidas pelo usuário, e utilizando elas de forma segura, sem risco de vazamento de dados críticos do sistema, como chaves de autenticação.

O Functions permite definir e usar variáveis de ambiente com o *Secret Manager* da nuvem do Google (Google LLC, 2025g).

Os gatilhos que o Functions permite cadastrar servem para qualquer operação de Criar, Atualizar, Ler e Deletar do Firestore (Google LLC, 2025h), que foram utilizados para o envio das notificações na criação de eventos de cães, do *Cloud Storage* (Google LLC, 2025i), que foram utilizados para armazenar as URLs das imagens do *Cloud Storage* nos documentos referentes a elas, e do banco de dados em tempo real (Google LLC, 2025j), que foram utilizados para atualizar o *chat* do sistema, o mapa de avistamentos e o valor arrecadado dos pedidos de ajuda financeiros.

Essas características fizeram com que o Functions se destacasse como uma das funcionalidades mais poderosas do Firebase, tornando-o crucial para o desenvolvimento do sistema.

Exemplos de códigos do Firebase Functions serão apresentados na seção 5.2, nas subseções 5.2.1, 5.2.2 e 5.2.3.

5.1.3.4 *Firebase Cloud Storage*

O *Cloud Storage* do Firebase é utilizado para armazenar arquivos e imagens de forma segura na nuvem do Firebase, providenciando URLs de acesso aos mesmos, que podem ser salvas no próprio Firestore do Firebase e posteriormente serem utilizadas para carregarem os arquivos armazenados. (Google LLC, 2025k)

Bancos de dados, como o Firestore, não são otimizados para armazenar arquivos binários grandes, os *blobs*. O *Cloud Storage* oferece um armazenamento de *blobs* de alta escalabilidade, segurança e disponibilidade.

Assim como o Firestore, o *Cloud Storage* possui um sistema de Regras (Google LLC, 2025l), que funciona da mesma maneira que o sistema de Regras do Firestore, podendo controlar todas as gravações de um *bucket* do Storage até operações de um arquivo específico do Storage.

5.1.4 **AbacatePay**

Devido ao seu foco no PIX, taxas fixas por transações e facilidade de implementação o AbacatePay foi o *gateway* de pagamento escolhido para processar as transações financeiras do sistema (Abacate Pay, 2025a).

O AbacatePay permite não só o recebimento de transações PIX mas também o envio das mesmas, possibilitando que usuários do sistema possam realizar todo o fluxo de doação e saque dentro do sistema em si.

O AbacatePay possui taxas de transações fixas (Abacate Pay, 2025b), cobrando apenas oitenta centavos fixos para cada transação recebida na conta do AbacatePay, taxa fixa de oitenta centavos para os primeiros vinte saques do mês, aumentando para dois reais e

cinquenta centavos a partir do vigésimo primeiro, e limitando o saque diurno em cinco mil reais e noturno em mil reais, possibilitando um aumento de limite mediante contato com o suporte.

Essas características tornaram o AbacatePay uma ótima escolha de *gateway* de pagamento inicial, com taxas aceitáveis, implementação rápida e fácil e também uma documentação limpa e intuitiva.

5.2 Desenvolvimento

Nesta seção serão apresentados os trechos de código das principais funcionalidades do Cãomunidade, desde as *functions* do Firebase referentes a integração com o **Expo Notifications** e o *gateway* de pagamento AbacatePay, até a forma que é renderizado o mapa de **Avistamentos** e a troca de **Mensagens** dos **Pedidos de Ajuda**.

Por se tratar de um *MVP* (*Minimum Viable Product* ou mínimo produto viável), o código ainda apresenta bastante espaço para melhorias de qualidade. O foco foi desenvolver as funcionalidades de uma maneira em que uma prova de conceito fosse concebida através de uma breve apresentação do que o sistema seria capaz enquanto produto.

5.2.1 Function de Criação de PIX do AbacatePay

Nesta subseção será apresentada a *Function* responsável por criar o PIX do AbacatePay e renderizar em tela o *QR Code* e o código copia e cola, para que o usuário realize o pagamento.

A seguir, no trecho de código 5.1, será detalhado como é criado o PIX chamando a API do AbacatePay.

```

1 export const abacatePayProxy = onCall({region: region}, async (request)
  => {
2   if (!request.auth) {
3     throw new HttpsError(
4       "unauthenticated",
5       "Você precisa estar logado para realizar esta ação.",
6     );
7   }
8
9   const {action, data} = request.data;
10
11   logger.info(`Ação recebida: ${action}`, {data});
12
13   switch (action) {
14     case "CREATE_CHARGE": {
15       if (!data.value || data.value <= 0) {
16         throw new HttpsError("invalid-argument", "O valor da doação é inv
          álido.");
17       }
18       try {
19         const response = await fetch(`${API_URL}/pixQrCode/create`, {
20           method: "POST",
21           headers: {
22             "Authorization": `Bearer ${ABACATEPAY_KEY.value()}`,
23             "Content-Type": "application/json",
24           },
25           body: JSON.stringify({amount: data.value}),
26         });
27         const responseData = await response.json();

```

```

28     if (!response.ok) {
29         logger.error("Erro AbacatePay (criar):", responseData);
30         throw new HttpsError("internal", "Erro do gateway de pagamento
31     .");
32     }
33     return responseData.data;
34 } catch (error) {
35     logger.error("Falha na chamada (criar):", error);
36     throw new HttpsError("internal", "Não foi possível gerar a cobran
37     ça.");
38 }

```

Código 5.1: Function de criação de PIX do AbacatePay

A *function* `abacatePayProxy` (linha 1 a 37) tem esse nome pois ela atua como um intermediário entre a API do AbacatePay e o sistema Cãomunidade, tratando a chamada que será feita para a API do AbacatePay, tanto para a criação do PIX como também para simular o processo de pagamento.

Nas linhas 2 a 7 é feito uma validação se o usuário está autenticado, logo após, caso a ação recebida seja a de criar pagamento (linha 14), é então chamado o trecho de código que cuida da lógica de criar um pagamento na API do AbacatePay.

Primeiramente, o valor é validado (linhas 15 a 17), após a validação, é feito um *try catch* encapsulando uma chamada para a API do AbacatePay (linhas 18 a 35). O *token* de autenticação (linha 22) é definido como uma variável *secret* do **Firestore** previamente e utilizado em código ao autenticar a chamada *POST* a API do AbacatePay.

Após o sucesso da chamada *POST*, é retornado então os dados da API do AbacatePay, que serão apresentados no trecho de código 5.3

O corpo da requisição, apresentado no trecho de código 5.2 é somente o valor da doação que será feita, que é enviado em centavos, seguindo as orientações de uso da API do AbacatePay.

```

1 {
2   "amount": 1000,
3   "description": "Doação teste Cãomunidade"
4 }

```

Código 5.2: Corpo da requisição para criar o PIX

Somente o atributo *amount* é obrigatório, tornando a criação de PIX via API do AbacatePay simples e fácil de utilizar.

```

1 {
2   "error": null,
3   "data": {
4     "amount": 1000,
5     "status": "PENDING",
6     "devMode": true,
7     "method": "PIX",
8     "brCode": "00020101021126580014BR.GOV.BCB.PIX0136devmode-pix-
9     pix_char_mac0baWUAJeCgnPPyCRuyFBr520400005303986540610.005802
10    BR5920AbacatePay DevMode6009Sao Paulo62070503***6304B14F",
11    "brCodeBase64": "...", // Base64 que gera a imagem do QR Code
12    do PIX
13    "platformFee": 80,
14    "description": "Doação teste Cãomunidade",
15    "createdAt": "2025-11-22T16:14:02.222Z",

```



```

13     "updatedAt": "2025-11-22T16:14:02.222Z",
14     "expiresAt": "2025-11-23T16:14:02.222Z",
15     "id": "pix_char_mac0baWUAJeCgnPPyCRuyFBr"
16   }
17 }

```

Código 5.3: Resposta de sucesso da criação do PIX

O retorno da requisição, em caso de sucesso, é um *JSON* com os dados do pix (linha 3 a 15), com o código copia e cola (linha 8, campo `brCode`), o *base64* da imagem do *QR Code* (linha 9, campo `brCodeBase64`),

5.2.2 Webhook de Confirmar Pagamento

Nesta subseção será detalhada toda a lógica em volta da *Function* que atua como um *webhook*, recebendo requisições da API do AbacatePay e confirmando o pagamento de um pix de doação para o sistema.

O AbacatePay permite o cadastro de *webhooks*, cada um com uma URL específica e eventos relacionados. O *webhook* utilizado no sistema Cãomunidade recebe apenas o evento de *billing.paid* (cobrança paga), que é enviado pelo AbacatePay sempre que um pagamento de uma cobrança, neste caso o PIX de doação, é efetuado. A seguir, no trecho de código 5.4, é possível ver um exemplo do corpo de requisição que é enviado pelo AbacatePay ao *webhook*.

```

1 {
2   "event": "billing.paid",
3   "data": {
4     "pixQrCode": {
5       "id": "pix_char_kgUct54sBsh0ATRMDFzcjcQL",
6       "amount": 250,
7       "kind": "PIX",
8       "status": "PAID"
9     },
10    "payment": {
11      "amount": 250,
12      "fee": 80,
13      "method": "PIX"
14    }
15  },
16  "devMode": true
17 }

```

Código 5.4: Corpo de requisição enviado ao Webhook

Ao receber um pagamento de uma cobrança, o AbacatePay envia ao *webhook* um corpo de requisição com os dados da mesma, como o tipo de pagamento no campo `method` (linha 13) dentro do objeto `payment` (linha 10), a quantia do pagamento `amount` (linha 11) e também o *id* do PIX que foi criado (linha 5).

A seguir, no trecho de código 5.5, é detalhado o início da *Function* que processa o corpo de requisição do *webhook*, desde o processo de autenticação até a atualização do *status* do pagamento no banco de dados Firestore.

```

1 export const abacatePayWebhook = onRequest({
2   region: region}, async (request, response) => {
3   if (request.method !== "POST") {
4     response.status(405).send("Method Not Allowed");
5     return;

```

```

6  }
7
8  const receivedSignature = request.headers["x-abacatepay-signature"]
   as string;
9  const webhookSecret = ABACATEPAY_WEBHOOK_SECRET.value();
10
11  if (!receivedSignature || !webhookSecret) {
12    logger.error("Webhook: Assinatura ou segredo ausente.");
13    response.status(400).send("Assinatura ausente.");
14    return;
15  }
16
17  try {
18    const hmac = crypto.createHmac("sha256", webhookSecret);
19    const digest = Buffer.from(
20      hmac.update(request.rawBody).digest("hex"), "utf8");
21    const checksum = Buffer.from(receivedSignature, "utf8");
22
23    if (checksum.length !== digest.length ||
24        !crypto.timingSafeEqual(digest, checksum)) {
25      logger.error("Webhook: Assinatura inválida.");
26      response.status(403).send("Assinatura inválida.");
27      return;
28    }
29    logger.info("Webhook: Assinatura verificada com sucesso!");
30  } catch (error) {
31    logger.error("Webhook: Erro ao verificar assinatura:", error);
32    response.status(500).send("Erro interno na verificação.");
33    return;
34  }
35
36  const eventPayload = request.body;
37  logger.info("Webhook recebido:", JSON.stringify(eventPayload));

```

Código 5.5: Autenticação da chave enviada pelo Webhook

A *Function* `abacatePayWebhook` (linha 1) é declarada como uma função `onRequest`, que é uma função que pode ser salva como uma URL que recebe requisições, nesse caso em específico do *webhook* do *AbacatePay*.

Primeiro, é validado o método da requisição (linhas 3 a 6), e então, se alguma chave secreta foi enviada (linhas 8 a 15). Se foi enviada alguma chave secreta, a função valida se a chave secreta que foi enviada é a mesma salva nas variáveis secretas do *Functions* (linha 17 a 34). Se a chave secreta enviada é igual á mesma salva no sistema, o corpo da requisição é salvo em uma variável (linha 36), para posteriormente ser processado pela função, conforme o trecho de código 5.6

```

1  if (
2    eventPayload.event === "billing.paid" &&
3    eventPayload.data?.payment?.method === "PIX" &&
4    eventPayload.data?.pixQrCode?.status === "PAID"
5  ) {
6    const chargeId = eventPayload.data.pixQrCode.id;
7    const valorPagoCentavos = eventPayload.data.payment.amount;
8    const valorPagoReais = valorPagoCentavos / 100;
9
10   if (!chargeId || valorPagoReais <= 0) {
11     logger.error("Webhook: chargeId ou valor inválido.", eventPayload
12   );

```

```

12     response.status(400).send("Payload inválido");
13     return;
14 }
15
16 logger.info(`Webhook: Pagamento recebido ${chargeId},
17     R$ ${valorPagoReais}.`);
18
19 const doacoesQuery = db.collectionGroup("doacoes")
20     .where("id_transacao_gateway", "==", chargeId)
21     .where("status", "==", "PENDENTE");
22
23 try {
24     const snapshot = await doacoesQuery.get();
25     if (!snapshot.empty) {
26         const doacaoDoc = snapshot.docs[0];
27         const doacaoRef = doacaoDoc.ref;
28         const pedidoRef = doacaoRef.parent.parent;
29
30         if (!pedidoRef) {
31             logger.error(`Webhook: Pedido pai não encontrado para
32                 ${doacaoDoc.id}`);
33             response.status(500).send("Erro hierarquia");
34             return;
35         }
36
37         await db.runTransaction(async (transaction) => {
38             transaction.update(doacaoRef, {
39                 status: "PAGO", data_pagamento: new Date(),
40             });
41             transaction.update(pedidoRef, {
42                 valor_arrecadado: admin.
43                     firestore.FieldValue.increment(valorPagoReais),
44             });
45         });
46
47         logger.info(`Webhook: Doação ${doacaoDoc.id}
48             OK no pedido ${pedidoRef.id}.`);
49     } else {
50         logger.warn(`Webhook: Nenhuma doação PENDENTE ${chargeId}.`);
51     }
52 } catch (error) {
53     logger.error(`Webhook: Erro ${chargeId}:`, error);
54     response.status(500).send("Internal Server Error");
55     return;
56 }
57 } else {
58     logger.info("Webhook: Evento ignorado.", eventPayload.event);
59 }
60 response.status(200).send("Received");

```

Código 5.6: Atualização de status de pagamento de um PIX

Ao receber o corpo da requisição do *Webhook*, a função valida se o evento recebido é o de cobrança paga e se o método é PIX e se foi pago (linhas 1 a 5), e então, em caso de sucesso, é iniciado o procedimento de validação das informações recebidas pelo corpo da requisição.

Primeiramente é validado se o *id* do PIX foi enviado e se o valor pago é maior que 0 (linhas 6 a 14).

Se caso sim, é feito uma consulta na coleção de doações, onde o *id* do PIX igual ao enviado pelo corpo da requisição e o *status* da doação seja "PENDENTE"(linhas 19 a 21).

Caso seja encontrado (linha 25), é validado se o pedido de ajuda pai daquela doação existe (linhas 28 a 35).

Em caso de sucesso, é atualizado o *status* da doação, e o valor arrecadado do pedido de ajuda também, com o valor novo (linhas 37 a 45).

5.2.3 Envio de Notificações

Nesta subseção será detalhado o processo de envio de notificações *push*, usando a API de notificações do Expo.

Para apresentar como é feito o processo de envio de notificações, serão apresentados três Functions, com uma delas sendo um gatilho do Firestore para quando um novo evento de cão é cadastrado, no trecho de código 5.7, e as outras duas funções assíncronas: uma para enviar as notificações, no trecho de código 5.9, e outra para buscar os *tokens* dos seguidores de um cão, no trecho de código 5.8

```

1 export const onNewEventForFollowers = functionsV1.region(region).
  firestore
2   .document("caes/{caoId}/eventos/{eventoId}")
3   .onCreate(async (snap, context) => {
4     const caoId = context.params.caoId;
5     const eventoData = snap.data();
6     const caoDoc = await db.collection("caes").doc(caoId).get();
7     const caoData = caoDoc.data();
8
9     if (!eventoData || !caoData) return null;
10
11    const followerTokens = await getFollowerTokens(caoId);
12
13    if (followerTokens.length > 0) {
14      return sendExpoPushNotification(followerTokens, {
15        title: `Novo evento para ${caoData.nome}!`,
16        body: `Evento: ${eventoData.descricao}.`,
17      });
18    }
19    return null;
20  });

```

Código 5.7: Gatilho do Firestore para quando um novo Evento de Cão é cadastrado

A *Function* `onNewEventForFollowers` é chamada sempre que um novo evento é cadastrado para um cão (linhas 1 a 3), recebendo os dados do evento (linha 5) e buscando os dados do cão, usando seu *id* (linhas 4,6 e 7).

Após validar se o evento e o cão existem (linha 9), a *Function* `getFollowerTokens`, apresentada no trecho de código 5.8, é chamada, para obter os *tokens* de notificação dos seguidores (linha 11), que são necessários para enviar as notificações *push* via API do Expo.

Após receber os *tokens* dos seguidores do cão, é chamada a *Function* `sendExpoPushNotification` detalhada no trecho de código 5.9, para enviar as notificações para os seguidores do cão sobre o novo evento cadastrado.

A seguir, será apresentada a *Function* `getFollowerTokens`, que é usada para buscar os *tokens* dos seguidores de um cão para serem enviadas as notificações *push* via API do Expo.

```

1 async function getFollowerTokens(caoId: string): Promise<string[]> {
2   const followersSnapshot = await db
3     .collection("users")
4     .where("caes_seguindo_ids", "array-contains", caoId)
5     .get();
6
7   if (followersSnapshot.empty) return [];
8   return followersSnapshot.docs
9     .map((doc) => doc.data().pushToken)
10    .filter((token) => token);
11 }

```

Código 5.8: Function usada para obter tokens de seguidores de um cão

A *Function* `getFollowerTokens` recebe o *id* de um cão somente (linha 1) e então busca (linhas 2 a 5) e retorna (linhas 8 a 10) os *tokens* de *push*, que são utilizados para enviar as notificações *push* via API do Expo.

A seguir, no trecho de código 5.9, será detalhado como é realizado o envio dessas notificações, chamando a API do Expo de envio de notificações *push*, passando os dados da notificação e os *tokens* de cada usuário que irá receber a notificação.

```

1 async function sendExpoPushNotification(pushToken: string | string[],
2   message: { title: string, body: string }) {
3   if (!pushToken || (Array.isArray(pushToken) && pushToken.length ===
4     0)) {
5     logger.info("Nenhum push token válido encontrado, pulando notificação.");
6     return;
7   }
8   const messages = (Array.isArray(pushToken) ? pushToken : [pushToken])
9     .map(
10      (token) => ({
11        to: token,
12        sound: "default",
13        title: message.title,
14        body: message.body,
15      })
16    );
17   try {
18     await fetch("https://exp.host/--/api/v2/push/send", {
19       method: "POST",
20       headers: {
21         "Accept": "application/json",
22         "Accept-encoding": "gzip, deflate",
23         "Content-Type": "application/json",
24       },
25       body: JSON.stringify(messages),
26     });
27     logger.info("Notificação enviada com sucesso para a API da Expo.");
28   } catch (error) {
29     logger.error("Erro ao enviar notificação para a API da Expo:",
30       error);
31   }

```

Código 5.9: Function de enviar notificações *push*

A *Function* `sendExpoPushNotification` recebe o *pushToken* e o conteúdo da notificação (*message*) (linhas 1 a 2). Logo após, é validado se são vários *pushTokens* ou um só, e se eles existem (linhas 3 a 6).

Em caso de sucesso, os *pushTokens* e o conteúdo das mensagens de notificação são salvos em uma variável *messages* (linhas 8 a 15), de forma que facilite o processo de envio de notificações.

Após isso, é feito a chamada da API de envio de notificações *push* do Expo (linhas 17 a 25), passando os *pushTokens* e o conteúdo das mensagens de notificação como corpo da requisição para a API, via método *POST*.

5.2.4 Renderização de Mensagens

Nesta subseção será detalhado como foi feito a sala de *chat* do sistema, que é disponibilizada após um pedido de ajuda ser cadastrado e algum usuário decidir ajudar ou doar, caso seja um pedido de ajuda financeiro. Começando pela declaração de dois componentes, conforme o trecho de código 5.10

```

1 const TypingIndicator = () => (
2   <View style={styles.typingContainer}>
3     <Text style={styles.typingText}>digitando...</Text>
4   </View>
5 );
6
7 const MessageBubble = ({ message, isMyMessage }) => {
8   const messageTime = message.createdAt?.toLocaleTimeString([], {
9     hour: '2-digit',
10    minute: '2-digit',
11  }) || '';
12
13   return (
14     <View style={[styles.messageRow, isMyMessage ? styles.
15       myMessageRow : styles.theirMessageRow]}>
16       <View style={[styles.messageBubble, isMyMessage ? styles.
17         myMessageBubble : styles.theirMessageBubble]}>
18         <Text style={isMyMessage ? styles.myMessageText :
19           styles.theirMessageText}>
20           {message.text}
21         </Text>
22         <Text style={isMyMessage ? styles.timestampMy : styles.
23           timestampTheir}>
24           {messageTime}
25         </Text>
26       </View>
27     </View>
28   );
29 };

```

Código 5.10: Início do código do Chat

O Componente `TypingIndicator` (linhas 1 a 5) é responsável por renderizar em tela para o usuário quando o outro usuário está digitando, já o componente `MessageBubble` (linhas 7 a 24) é responsável por renderizar as mensagens do *chat* dinamicamente.

Em seguida, no trecho de código 5.11, temos a declaração dos estados utilizado na tela do *chat* e o primeiro *useEffect*, que é um *hook* do **React Native**, responsável por rodar uma função depois da montagem de um componente ou sempre que uma de suas dependências, que são passadas no segundo parâmetro da função como um *array*, mudam.

```

1 const ChatRoomScreen = ({ route }) => {
2   const { chatId } = route.params;
3   const [messages, setMessages] = useState([]);
4   const [inputText, setInputText] = useState('');
5   const [otherUserIsTyping, setOtherUserIsTyping] = useState(false);
6   const auth = getAuth();
7   const user = auth.currentUser;
8   const flatListRef = useRef(null);
9   const typingTimeoutRef = useRef(null);
10
11
12   useEffect(() => {
13     const messagesRef = collection(db, `chats/${chatId}/messages`);
14     const q = query(messagesRef, orderBy("createdAt", "desc"));
15
16     const unsubscribe = onSnapshot(q, (snapshot) => {
17       const messagesData = snapshot.docs.map(doc => {
18         const data = doc.data();
19         return {
20           id: doc.id,
21           text: data.text,
22           createdAt: data.createdAt?.toDate() || new Date(),
23           userId: data.userId,
24         };
25       });
26       setMessages(messagesData);
27     });
28
29     return () => unsubscribe();
30   }, [chatId]);

```

Código 5.11: Declaração de estados e primeiro `useEffect` do Chat

As variáveis de estado (linhas 2 a 9) são usadas, respectivamente, para guardar o *id* do *chat* (linha 2), que vem como um dos parâmetros de rota da tela anterior ao *chat*, as mensagens do *chat* (linha 3), o texto que o usuário está digitando (linha 4), um *booleano* que decide se o `TypingIndicator` deve aparecer (linha 5), a autenticação (linha 6), que é para conferir se o usuário está logado, o usuário em si (linha 7), e duas referências (linhas 8 e 9), uma para a lista de mensagens e outra para controlar o *timeout* que controla a exibição do `TypingIndicator`.

Em seguida, temos o primeiro *hook useEffect*, que utiliza do `onSnapshot` do banco de dados em tempo real do **Firestore** para "escutar" a coleção de mensagens do *chat* atual, ordenando as mensagens de forma decrescente pela data que foram criadas.

O `onSnapshot` atualiza as mensagens renderizadas em tela, mapeando o que é retornado pela consulta no banco de acordo com o que deve ser enviado para o componente `MessageBubble`, representado no trecho de código 5.10 anteriormente, e para a lista de mensagens, que será apresentada posteriormente.

A função `unsubscribe`, retornada pelo `onSnapshot`, é utilizada no retorno do *useEffect* para garantir que quando o usuário saia da tela o sistema deixe de escutar a coleção de mensagens, economizando o consumo do banco de dados.

A seguir, no trecho de código 5.12, é apresentado o *hook* de efeito que renderiza o componente `TypingIndicator` e a função chamada quando o usuário está digitando.

```

1 useEffect(() => {
2   const chatRef = doc(db, 'chats', chatId);
3   const unsubscribe = onSnapshot(chatRef, (doc) => {

```

```

4         const data = doc.data();
5         if (data?.typing && data.typing.userId !== user.uid && data
        .typing.isTyping) {
6             setOtherUserIsTyping(true);
7         } else {
8             setOtherUserIsTyping(false);
9         }
10    });
11    return () => unsubscribe();
12 }, [chatId, user.uid]);
13
14 const handleTyping = (isTyping) => {
15     const chatRef = doc(db, 'chats', chatId);
16     updateDoc(chatRef, {
17         typing: {
18             isTyping: isTyping,
19             userId: user.uid,
20         }
21     });
22 }

```

Código 5.12: Segundo *useEffect* do Chat e função chamada ao digitar mensagem

O segundo *useEffect* usa o *onSnapshot* também, mas para escutar a coleção do *chat* em si, e não de suas mensagens. O *onSnapshot* confere se o campo *isTyping* é verdadeiro ou falso e se o *uid* que está salvo no *chat* é o mesmo do usuário atual, conforme o retorno dessas condições, ele chama a função que controla o estado da renderização do *TypingIndicator*, que é a função *setOtherUserIsTyping*.

A função *setOtherUserIsTyping* é uma função que controla o estado da variável de estado *otherUserIsTyping*, que usa o *hook useState*.

A seguir, no trecho de código 5.13, está a função chamada ao clicar no botão de enviar mensagem e a função que realiza as lógicas necessárias para gerenciar a digitação do texto do usuário.

```

1 const onSend = useCallback(async () => {
2     const text = inputText.trim();
3     if (text.length === 0) {
4         return;
5     }
6
7     setInputText('');
8     handleTyping(false);
9
10
11     const messagesRef = collection(db, `chats/${chatId}/messages`);
12     await addDoc(messagesRef, {
13         text,
14         createdAt: serverTimestamp(),
15         userId: user.uid,
16     });
17
18     const chatRef = doc(db, 'chats', chatId);
19     await updateDoc(chatRef, {
20         lastMessage: {
21             text,
22             timestamp: serverTimestamp(),
23             sentBy: user.uid,
24         },

```



```

25     });
26   }, [chatId, inputText, user]);
27
28   const handleInputChange = (text) => {
29     setInputText(text);
30
31     if (typingTimeoutRef.current) {
32       clearTimeout(typingTimeoutRef.current);
33     }
34
35     handleTyping(true);
36
37     typingTimeoutRef.current = setTimeout(() => {
38       handleTyping(false);
39     }, 3000);
40   };

```

Código 5.13: Funções de envio e de gerenciamento de estado de texto digitado pelo usuário

A função `onSend` verifica se o texto é vazio, e logo após, muda o estado do texto digitado e da renderização do componente `TypingIndicator`, em seguida, é feita uma inserção de um documento novo na coleção de mensagens, com o texto digitado, quem foi o usuário que enviou, e quando o texto foi enviado. Logo após, o documento do *Chat* é atualizado, com as informações da última mensagem.

A função `handleInputChange` é chamada sempre que uma nova letra é digitada pelo usuário, para atualizar o estado do texto digitado (linha 29) e para chamar a função que renderiza o componente `TypingIndicator` em tela. Ela cria também um *timer* (linhas 37 a 40), que renderiza o componente `TypingIndicator` por três segundos. Para evitar que o componente seja escondido enquanto o usuário ainda está digitando, qualquer *timer* criado anteriormente é limpo (linhas 31 a 33).

A seguir, no trecho de código 5.14, é apresentado o código dos componentes que são renderizados em tela para o usuário, como a lista de mensagens e o *input* de texto.

```

1  return (
2    <SafeAreaView style={styles.container}>
3      <KeyboardAvoidingView
4        style={styles.container}
5        behavior={Platform.OS === "ios" ? "padding" : "height"}
6        keyboardVerticalOffset={Platform.OS === "ios" ? 60 : 0}
7      >
8        <FlatList
9          ref={flatListRef}
10         style={styles.messageList}
11         data={messages}
12         renderItem={({ item }) => <MessageBubble message={
item} isMyMessage={item.userId === user.uid} />}
13         keyExtractor={item => item.id}
14         inverted
15       />
16
17       {otherUserIsTyping && <TypingIndicator />}
18
19       <View style={styles.inputContainer}>
20         <TextInput
21           style={styles.textInput}
22           value={inputText}

```

```

23             onChangeText={handleInputChange}
24             placeholder="Digite sua mensagem..."
25             placeholderTextColor={COLORS.grey}
26         />
27         <TouchableOpacity style={styles.sendButton} onPress
= {onSend}>
28             <FontAwesome5 name="paper-plane" size={22}
color={COLORS.white} solid />
29         </TouchableOpacity>
30     </View>
31 </KeyboardAvoidingView>
32 </SafeAreaView>
33 );
34 };

```

Código 5.14: Código dos componentes que são renderizados em tela

O `SafeAreaView` e o `KeyboardAvoidingView` (linhas 2 e 3) são utilizados para evitarem que a tela não fique atrás da barra do topo do celular e nem atrás da barra de gestos, que fica na parte de baixo da tela do celular, e que o teclado não cubra o *input* de texto, possibilitando que o usuário enxergue o que está digitando.

O componente `FlatList` (linha 8) é responsável renderizar as mensagens em tela, ele recebe as mensagens como os dados da lista (linha 11) e renderiza elas como o componente `MessageBubble` (linha 12), a propriedade *inverted* (linha 14) é utilizada para que a lista seja renderizada de baixo para cima, ao invés de como uma lista é tradicionalmente renderizada (de cima para baixo).

6 TESTES

Neste capítulo serão apresentados os testes realizados para comprovar que o sistema atingiu seus objetivos, como o cadastro de um Cão, o alerta de Cão desaparecido e as doações. Será detalhado todo o processo de realização dessas ações como também o tempo de resposta de cada.

Também será apresentado o aplicativo funcionando simultaneamente em ambas as plataformas móveis principais, que são Android e iOS.

6.1 Cadastro de um Cão

Nesta seção será realizado o teste de um cadastro de um Cão, visando confirmar o registro do mesmo no banco de dados do Firestore, como também demonstrar que o Cão está de fato cadastrado e sendo exibido na listagem de Cães do sistema.

Apresentado na Figura 6.1 está o formulário de cadastro de um novo Cão, já preenchido com as informações necessárias para cadastrar um Cão no sistema.



10:56

Cadastro Cão

Nome:
Phoebe

Fotos:
 +

Descrição:
teste phoebe

Gênero:
☐ Macho ☒ Fêmea

Porte:
☐ Pequeno ☒ Médio ☐ Grande

Histórico médico:
Vacinada

Tags:
adotado X

Adicione tags (ex: dócil) +

Figura 6.1: Formulário de cadastro de um Cão preenchido

Após a realização do cadastro, o sistema leva o usuário para a tela do perfil do novo

cão, mostrando em tela os dados do cão recém cadastrado, conforme é apresentado na Figura 6.2.

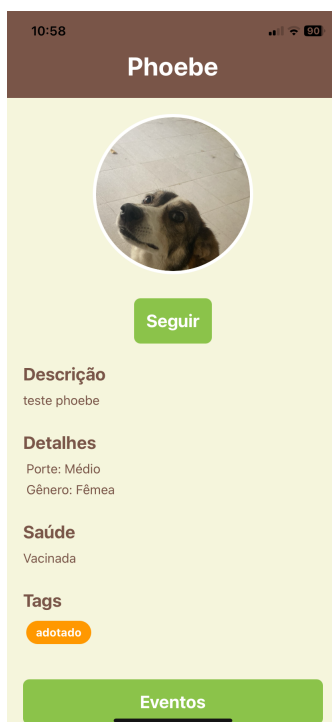


Figura 6.2: Perfil do novo Cão cadastrado

Após cadastrado, o novo cão é listado na tela de Cães, conforme a Figura 6.3, sendo possível acessar o seu perfil pela listagem de Cães.

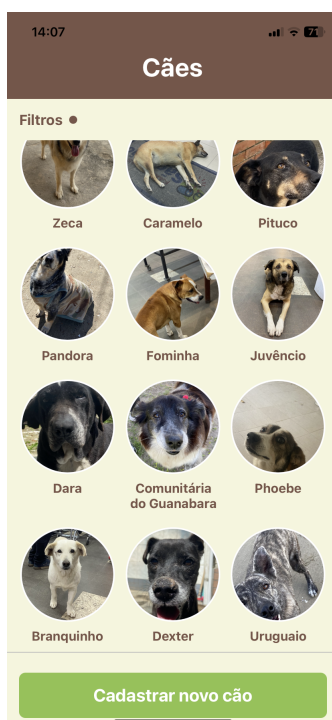


Figura 6.3: Listagem de Cães com o novo Cão cadastrado

Na Figura 6.4 é possível ver como o novo cão é salvo no banco de dados do Firestore, dentro da coleção **caes**, contendo seus respectivos campos referentes as suas informações, como a data de seu cadastro, seu nome, sua descrição, suas fotos, histórico médico, palavras-chave do nome, que são usadas para facilitar o filtro por um cão, e suas *tags*.

```

data_cadastro: 1 de dezembro de 2025 às 10:56:18 UTC-3
descricao: "teste phoebe"
foto_principal_url: "https://firebasestorage.googleapis.com/v0/b/caomunidade-71213.firebaseio.com/o/caes%2F1764597376258-0.pnubut8dp1?alt=media&token=266a9cc0-9f6a-4c8e-bc24-01c6313637d7"
fotos_urls:
  0 "https://firebasestorage.googleapis.com/v0/b/caomunidade-71213.firebaseio.com/o/caes%2F1764597376258-0.pnubut8dp1?alt=media&token=266a9cc0-9f6a-4c8e-bc24-01c6313637d7"
genero: "Fêmea"
historico_medico: "Vacinada"
id_usuario_criador: "xLOsPTK7R0bW2ZHOsgHVFL4wgyC3"
keywords_nome:
  0 "phoebe"
localizacao_atual: [32.181327883118065° S, 52.152249126224866° W]
nome: "Phoebe"
porte: "Médio"
status: "normal"
tags:
  0 "adotado"

```

Figura 6.4: Registro do novo cão no Firestore

6.2 Alerta de Cão Desaparecido

Nesta seção será realizado o teste de disparo de alerta de Cão Desaparecido, que é enviado para seguidores de um cão assim que o Evento de Cão Desaparecido é cadastrado para o cão.

Para receber o alerta, é necessário estar seguindo o cão que é relatado como desaparecido. Conforme a Figura 6.5, após o evento de Cão Desaparecido ser cadastrado, os usuários que seguem o cão são notificados sobre o evento de seu desaparecimento.



Figura 6.5: Evento de Cão Desaparecido e notificação sobre

No sistema, cães que estão desaparecidos tem seus perfis destacados com um *label* "DESAPARECIDO", conforme a Figura 6.6 apresenta.

Os cães que são relatados como desaparecidos tem seu *status* salvo no banco de dados como "desaparecido", conforme consta na Figura 6.7. O *status* serve para poder ajudar em filtros utilizados pelo sistema para buscar cães desaparecidos e destacar o seu perfil e seus avistamentos.

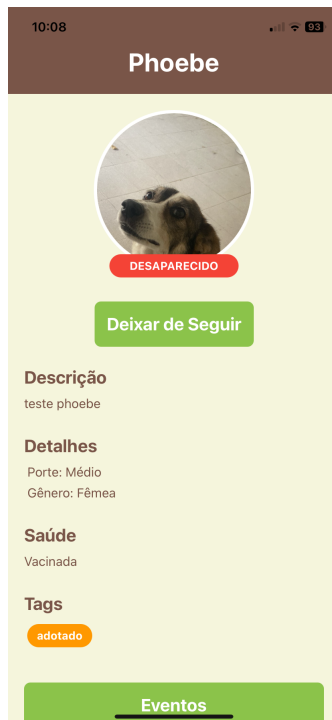


Figura 6.6: Perfil de um cão desaparecido

```
data_cadastro: 1 de dezembro de 2025 às 10:56:18 UTC-3
descricao: "teste phoebe"
foto_principal_url: "https://firebasestorage.googleapis.com/v0/b/caomunidade-71213.firebaseio.com/o/2F1764597376258-0.pnubut8dp1?alt=media&token=266a9cc0-9f6a-4c8e-bc24-01c6313637d7"
fotos_urls:
  - "https://firebasestorage.googleapis.com/v0/b/caomunidade-71213.firebaseio.com/o/2F1764597376258-0.pnubut8dp1?alt=media&token=266a9cc0-9f6a-4c8e-bc24-01c6313637d7"
genero: "Fêmea"
historico_medico: "Vacinada"
id_usuario_criador: "xLOsPTK7R0bW2ZH0sgHVFL4wgyC3"
keywords_nome:
  - "phoebe"
nome: "Phoebe"
porte: "Médio"
status: "desaparecido"
tags:
  - "adotado"
```

Figura 6.7: Registro do novo cão no Firestore

Para um cão perder seu *status* de desaparecido é necessário que seja cadastrado um avistamento sobre o mesmo. O cadastro de um avistamento será detalhado na seção 6.3.

6.3 Cadastro de um Avistamento

Nesta seção será detalhado o cadastro de um avistamento, usado para identificar pontos aonde um cão foi visto, o que auxilia na localização e rastreio de cães como também a identificar cães que estavam desaparecidos.

Primeiramente, é necessário acessar a tela de avistamentos, conforme mostra a Figura 6.8, contendo o mapa de avistamentos e um botão de registrar um avistamento.

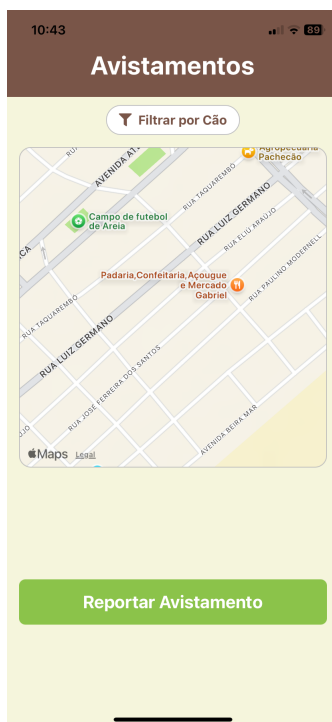


Figura 6.8: Tela de Avistamentos

Ao clicar em registrar um novo avistamento, o sistema pede permissão a câmera do usuário, e após a permissão ser concedida, o sistema abre a câmera, contendo um botão de tirar a foto para registrar o avistamento.

Após tirar a foto, o sistema leva o usuário para o formulário de avistamento, conforme a Figura 6.9, contendo as informações de data e horário do avistamento, como também a opção de relatar qual é o cão do avistamento, possibilitando que o usuário selecione um cão entre os cadastrados no sistema, ou marque que é um cão desconhecido.

Após registrar o avistamento, ele é exibido no mapa, conforme a Figura 6.10, possibilitando que o usuário clique no marcador e confira a data e a foto do avistamento.

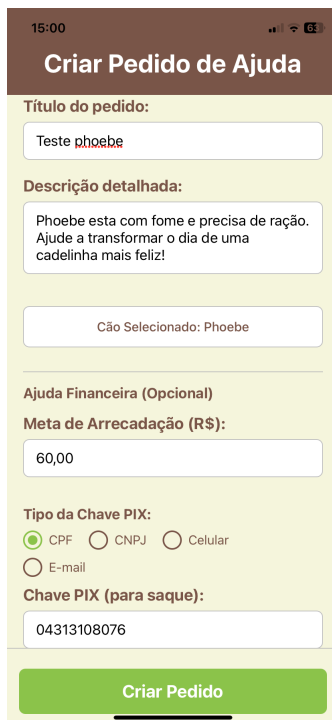
Figura 6.9: Registro do Avistamento de um cão

Figura 6.10: Registro do Avistamento de um cão

6.4 Pedido de Ajuda Financeiro

Nesta seção será detalhado o cadastro de um Pedido de Ajuda Financeiro, e também como é feita uma doação para o mesmo, como ela é salva no banco de dados e como é a prestação de contas de um pedido de ajuda financeiro.

Primeiramente, é necessário cadastrar um pedido de ajuda, que é salvo como um pedido de ajuda financeiro caso seja informado uma meta de dinheiro para arrecadação e uma chave PIX para sacar o dinheiro do pedido de ajuda, conforme apresenta a Figura 6.11.



15:00

Criar Pedido de Ajuda

Título do pedido:

Teste phoebe

Descrição detalhada:

Phoebe esta com fome e precisa de ração. Ajude a transformar o dia de uma cadelinha mais feliz!

Cão Selecionado: Phoebe

Ajuda Financeira (Opcional)

Meta de Arrecadação (R\$):

60,00

Tipo da Chave PIX:

☒ CPF ☐ CNPJ ☐ Celular

☐ E-mail

Chave PIX (para saque):

04313108076

Criar Pedido

Figura 6.11: Formulário de cadastro de um pedido de ajuda

Após cadastrado o pedido de ajuda financeiro é possível que o criador realize uma série de ações com o pedido, conforme mostra a Figura 6.12, como sacar o dinheiro do pedido, caso haja algum, marque ele como resolvido ou preste contas, afim de mostrar onde o dinheiro arrecadado pelo pedido foi usado.

Usuários que não são criadores de um pedido de ajuda financeiro tem as opções de doar ou visualizar a prestação de contas de um pedido de ajuda financeiro, conforme é apresentado na Figura 6.13.

Ao clicar em "Doar", o sistema abre um *modal* (componente visual que sobrepõe a tela), que pode ser visto na Figura 6.14, com um campo de valor, onde o doador pode especificar quanto querdoar para a campanha.

Após gerado o PIX de doação, o sistema traz em tela o código "copia e cola" e um *QR Code*, usados para que o usuário copie o código ou acesse o *QR Code* a partir de uma câmera e pague usando seu aplicativo de banco de preferência.

Ao gerar o PIX de doação, uma cobrança é criada no AbacatePay, conforme mostra a Figura 6.15 essa cobrança é criada com o *status* Pendente, até que seja paga.

Afim de rastrear as doações que foram criadas e trazer os dados de arrecadação em tempo real e corretamente, o sistema também salva a cobrança no banco de dados, conforme demonstrado na Figura 6.16.

Após realizado o pagamento do PIX de doação, a cobrança é atualizada no AbacatePay, com o *status* de Sucesso, apresentado na Figura 6.17.

Ao confirmar o pagamento de uma cobrança, o AbacatePay dispara um *webhook*, que foi apresentado na subseção 5.2.2. Esse *webhook* é recebido pelo sistema e atualiza o



Figura 6.12: Pedido de ajuda financeiro - Visão do criador do pedido



Figura 6.13: Pedido de ajuda financeiro - Visão do doador do pedido

status da doação, com o *status* "Pago", como mostra a Figura 6.18.

E por fim, o criador do pedido de ajuda financeiro pode prestar contas, fornecendo fotos que comprovam onde foi usado o dinheiro. As fotos podem ser tanto comprovantes ou notas fiscais de compra de medicamentos ou pagamentos de consultas com veterinário, como também podem ser fotos de animais no pós-operatório. No exemplo da Figura

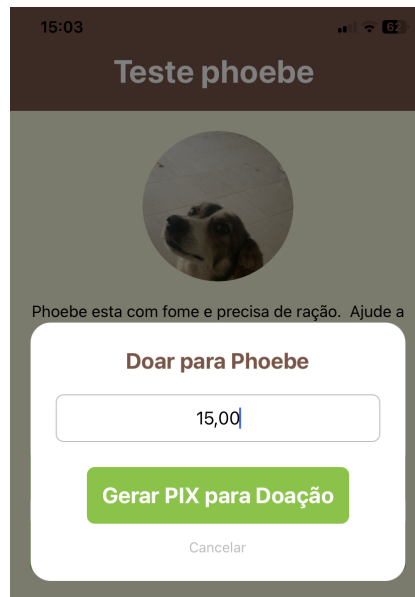


Figura 6.14: Modal de doação

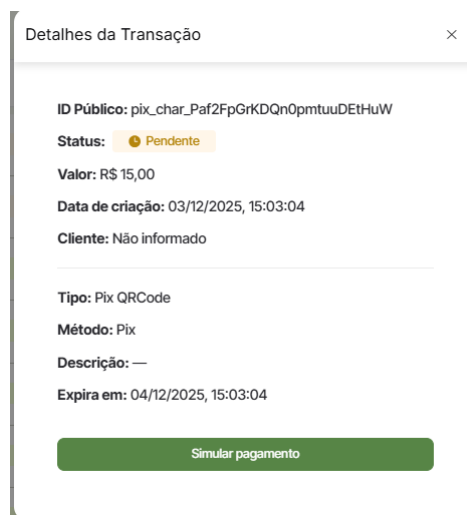


Figura 6.15: Cobrança pendente no AbacatePay

```
data_criacao: 3 de dezembro de 2025 às 15:03:04 UTC-3
id_transacao_gateway: "pix_char_Paf2FpGrKDQnOpmtuuDEtHuW"
id_usuario_doador: "TTuFLTNDuNRz0Zydc9uuGvCwfWY2"
status: "PENDENTE"
valor: 15
```

Figura 6.16: Doação pendente no Firestore

6.19 temos o cão Phoebe com a ração que foi comprada com o dinheiro arrecadado na campanha.

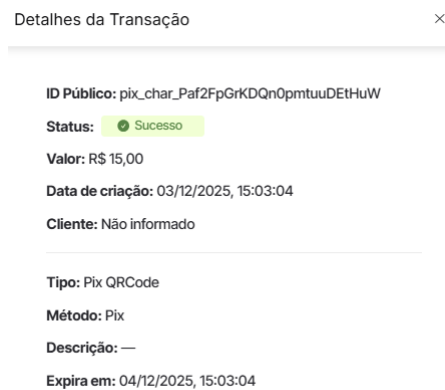


Figura 6.17: Cobrança paga no AbacatePay

```
data_criacao: 3 de dezembro de 2025 às 15:03:04 UTC-3
data_pagamento: 3 de dezembro de 2025 às 15:05:57 UTC-3
id_transacao_gateway: "pix_char_Paf2FpGrKDQnOpmtuuDEtHuW"
id_usuario_doador: "TTuFLTNDuNRz0Zydc9uuGvCwfWY2"
status: "PAGO"
valor: 15
```

Figura 6.18: Doação paga no Firestore

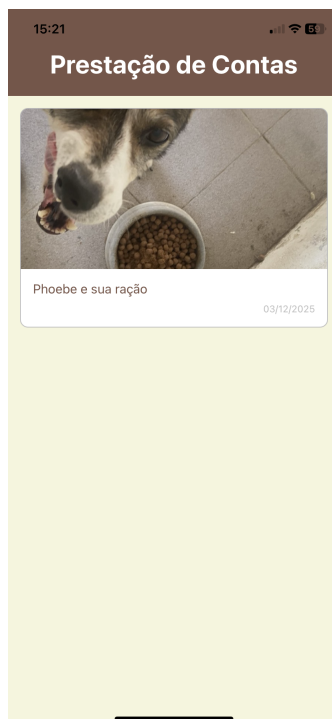


Figura 6.19: Prestação de contas do pedido de ajuda financeiro

6.5 Multiplataforma

Nesta seção será apresentado o sistema funcionando em ambas plataformas Android e iOS. Será usado como exemplo uma interação entre criador e ajudante de um pedido de ajuda não financeiro, onde o usuário que quiser ajudar com o pedido é levado para uma

sala de *chat* com o criador do mesmo.

Primeiramente, o usuário na plataforma Android acessa um pedido de ajuda não financeiro e clica no botão "Ajudar", conforme a Figura 6.20. O usuário criador do pedido de ajuda então recebe uma notificação de que alguém está disposto a ajudar no seu pedido de ajuda.



Figura 6.20: Usuário na plataforma Android acessando um pedido de ajuda

O usuário criador do pedido de ajuda, que está na plataforma iOS, troca mensagens com o usuário ajudante do pedido de ajuda, que está na plataforma Android.

A visão do usuário do iOS é apresentada na Figura 6.21, e a do usuário Android na Figura 6.22.

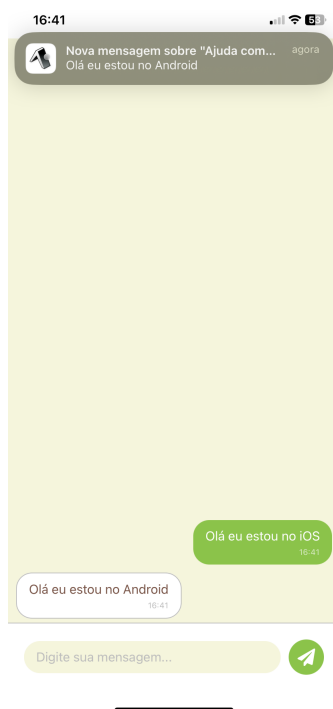


Figura 6.21: Usuário na plataforma iOS trocando mensagens no chat com o usuário da plataforma Android

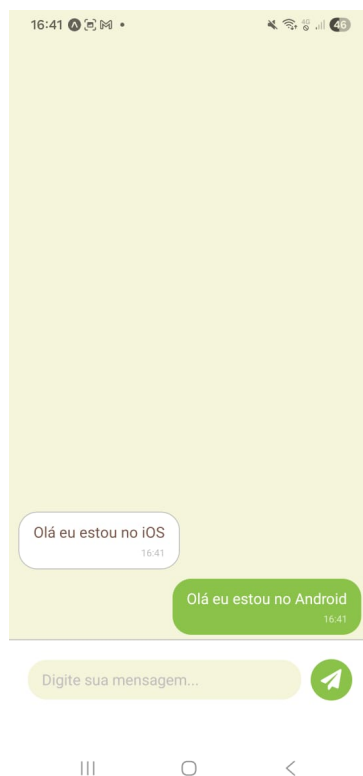


Figura 6.22: Usuário na plataforma Android trocando mensagens no chat com o usuário da plataforma iOS

7 CONCLUSÃO

O Cãomunidade surgiu de uma necessidade de centralizar os cuidados dos cães comunitários, visto que o cuidado atual dos mesmos é organizado na medida do possível mas em grande parte sofre com a carga enorme de trabalho que provém da responsabilidade de manter os cães em plenas condições.

Através de uma entrevista foi possível levantar os requisitos necessários para que o sistema atingisse seus objetivos com sucesso e excelência. Foi analisado cada caso de uso que um usuário encontrari ao interagir com o sistema, de modo que fosse atendido as expectativas do que o sistema fosse capaz.

A prototipação do sistema foi feita de maneira objetiva, focando em implementar uma aplicação leve e ágil, que providenciassse uma solução multiplataforma acessível e de alto impacto na causa social dos cuidados de cães comunitários.

As ferramentas utilizadas se provaram suficientes para atingir os objetivos do projeto, com o Firebase providenciando uma gama de funcionalidades com sua arquitetura *serverless* tornando o aplicativo rápido e de baixa complexidade técnica. O AbacatePay sendo um *gateway* de pagamento inicial que já abrange suficientemente os requisitos financeiros do aplicativo. O React Native agindo como uma sólida solução para o objetivo de multiplataforma, provendo um código de *frontend* compatível com ambas as principais plataformas do mercado, que são Android e iOS. E por fim o Expo, auxiliando com o desenvolvimento do projeto e fornecendo bibliotecas valiosas, com a de envio de notificações.

A tabela 7.1 ressalta que o aplicativo obteve sucesso em cada um dos aspectos analisados na seção 2.4, conseguindo fornecer de forma centralizada uma amplitude de funcionalidades, descartando a necessidade de acessar cada aplicativo para uma necessidade específica.

Tabela 7.1: Comparativo entre os sistemas existentes e o Cãomunidade

Funcionalidade	Pet Coletivo	Viu meu Pet	Vakinha	Cãomunidade
Plataforma	Web / Android	Web	Web	Android e iOS
Cadastro individual de Cães	Não	Não	Não	Sim
Mapa de Avistamentos	Não	Não	Não	Sim
Campanhas Financeiras	Sim	Não	Sim	Sim
Prestação de Contas	Sim	Não*	Sim	Sim
Alertas de Desaparecimento	Não	Sim	Não	Sim
Chat Integrado	Não	Não	Não	Sim

*Funcionalidade prevista mas não implementada no sistema citado).

Conforme os testes vistos na seção 6, é seguro dizer que o projeto obteve sucesso em

atingir seus objetivos específicos, como as campanhas de arrecadação, os cadastros individuais de cães, os Avistamentos, os pedidos de ajuda não financeiros, o *chat* integrado e os eventos de cães.

Por se tratar de um MVP, o sistema abre também espaço para melhorias futuras e novas funcionalidades, como o saque, que não foi implementado por se tratar um sistema em fase de desenvolvimento, utilizando a versão de testes do *gateway* de pagamento do AbacatePay.

O Firestore, mesmo sendo um banco de dados *NoSQL* bastante eficaz e robusto, também possui suas limitações, como os documentos que não podem ultrapassar um *megabyte* de tamanho. Além de que o Firebase e todas suas funcionalidades, como o próprio Firestore e Functions, são pagos para utilizar, com modelos econômicos satisfatórios, mas que com um uso de longo prazo podem criar uma dívida significativa, tornando mais caro manter a aplicação.

Até mesmo o uso de *IA*(Inteligência Artificial) pode ser considerado uma melhoria futura, providenciando uma maneira de reconhecer individualmente cada cão cadastrado por meio de uma série de fotos, com um modelo treinado para o reconhecimento individual de cães. Algo que até foi considerado para a implementação do projeto, mas, devido a alta complexidade de reconhecimento de padrões de cães *SRD* (Sem Raça Definida) que definem a maioria dos cães comunitários, foi descartado o uso de *IA* em primeiro momento.

Contudo, é estimado que o Cãomunidade seja uma plataforma de grande apoio aos tutores, voluntários e cuidadores de cães comunitários, fornecendo uma solução inteligente e de alta capacidade, sendo capaz de auxiliar na arrecadação de fundos, rastreo e colecionamento de informações de cães comunitários na cidade de Rio Grande.

REFERÊNCIAS

Abacate Pay. **Abacate Pay - Documentação Oficial**. Acessado em: 29/10/2025, disponível em: <https://docs.abacatepay.com/>.

Abacate Pay. **Abacate Pay - Termos e condições**. Acessado em: 13/11/2025, disponível em: <https://www.abacatepay.com/termosmodo-de-producao>.

Expo. **Expo - Make any app. Run it everywhere**. Acessado em: 29/10/2025, disponível em: <https://expo.dev/>.

Expo. **Expo Notifications**. Acessado em: 29/10/2025, disponível em: <https://docs.expo.dev/versions/latest/sdk/notifications/>.

Google LLC. **Firebase - Google's Mobile Development Platform**. Acessado em: 29/10/2025, disponível em: <https://firebase.google.com/>.

Google LLC. **Firebase Authentication**. Acessado em: 29/10/2025, disponível em: <https://firebase.google.com/docs/auth>.

Google LLC. **Cloud Firestore - Firebase**. Acessado em: 29/10/2025, disponível em: <https://firebase.google.com/docs/firestore>.

Google LLC. **Realtime Database - Firebase**. Acessado em: 11/11/2025, disponível em: <https://firebase.google.com/docs/database?hl=pt-br>.

Google LLC. **Firestore - Regras - Firebase**. Acessado em: 13/11/2025, disponível em: <https://firebase.google.com/docs/firestore/security/rules-structure?hl=pt-br>.

Google LLC. **Cloud Functions for Firebase**. Acessado em: 29/10/2025, disponível em: <https://firebase.google.com/docs/functions>.

Google LLC. **Cloud Functions for Firebase - Configurar o ambiente - Parâmetros de Secret**. Acessado em: 12/11/2025, disponível em: <https://firebase.google.com/docs/functions/config-env?hl=pt-br#node.js7>.

Google LLC. **Cloud Functions for Firebase - Gatilhos do Firestore**. Acessado em: 12/11/2025, disponível em: <https://firebase.google.com/docs/functions/1st-gen/firestore-events-1st?hl=pt-br>.

Google LLC. **Cloud Functions for Firebase - Gatilhos do Cloud Storage**. Acessado em: 12/11/2025, disponível em: <https://firebase.google.com/docs/functions/1st-gen/gcp-storage-events-1st?hl=pt-br>.

Google LLC. **Cloud Functions for Firebase - Gatilhos do Realtime Database**. Acessado em: 12/11/2025, disponível em: <https://firebase.google.com/docs/functions/1st-gen/database-events-1st?hl=pt-br>.

Google LLC. **Cloud Storage for Firebase**. Acessado em: 29/10/2025, disponível em: <https://firebase.google.com/docs/storage>.

Google LLC. **Cloud Storage for Firebase - Regras de segurança**. Acessado em: 29/10/2025, disponível em: <https://firebase.google.com/docs/storage/security/core-syntax?hl=pt-br>.

Mars Petcare. **The State of Pet Homelessness Project**. Acessado em: 05/12/2025, disponível em: <https://stateofpethomelessness.com/latest-report/?Country=Brazil>.

Meta Platforms, Inc. **React Native – A framework for building native apps using React**. Acessado em: 29/10/2025, disponível em: <https://reactnative.dev/>.

Meta Platforms, Inc. **React Native – Core Componentes and Native Components**. Acessado em: 10/11/2025, disponível em: <https://reactnative.dev/docs/intro-react-native-components>.

Pet Coletivo. **Pet Coletivo - O site de vaquinha dos animais**. Acessado em: 11/09/2025, disponível em: <https://www.petcoletivo.com.br/>.

Pet Coletivo. **Pet Coletivo - Quanto Custa**. Acessado em: 11/09/2025, disponível em: <https://www.petcoletivo.com.br/Políticas/QuantoCusta>.

PRESSMAN, R. S.; MAXIM, B. R. **Engenharia de Software: uma abordagem profissional**. 9.ed. [S.l.]: McGraw-Hill, 2021.

React Native Community. **React Native Maps - React Native Mapview component for iOS + Android**. Acessado em: 29/10/2025, disponível em: <https://github.com/react-native-maps/react-native-maps>.

React Navigation Contributors. **React Navigation - Routing and navigation for Expo and React Native apps**. Acessado em: 29/10/2025, disponível em: <https://reactnavigation.org/>.

Vakinha. **Vakinha - Vaquinhas online**. Acessado em: 11/09/2025, disponível em: <https://www.vakinha.com.br/>.

Viu meu Pet. **Viu meu Pet**. Acessado em: 11/09/2025, disponível em: <https://www.viumeupet.com.br/>.